Technische Hochschule Augsburg

Faculty of Computer Science

Bachelor's Thesis

to obtain the degree

Bachelor of Science (B.Sc.) in International Information Systems

Development and Evaluation of Machine Learning Models for Lithium-Ion Battery Remaining Useful Life Prediction

created by

Nayeem Faisal

First Supervisor	Prof. Dr. rer. nat. Jana Görmer-Redding
Second Supervisor	Prof. Dr. Fiorentino Valerio Conte
Submitted on	27 October 2026
Non-Disclosure Agreement	No

Nayeem Faisal

Enrolment Number: 2224981

International Information Systems

Technische Hochschule Augsburg

An der Hochschule 1

D-86161 Augsburg

www.tha.de



FINAL CANDIDATE - AUTHOR AND SUPERVISOR APPROVAL REQUIRED BEFORE SUBMISSION

ABSTRACT

Lithium-ion batteries have emerged as a fundamental element of power supply systems, and the ability to precisely forecast their remaining useful life is critical for ensuring their dependability. Due to the intricate operational properties of lithium-ion batteries and the dynamic nature of model parameters that follow the ageing process, enhancing the precision of the online remaining useful life prediction is not only challenging but also imperative for applications requiring a dependable power supply. The aim of this research starts by optimizing the hyperparameters of LSTM addressing the challenge of manually tuning parameters and enhancing the algorithm's capability to escape local optima. Second, Synthesizer, a powerful tool for generating synthetic data is adopted. We specified distributions for numerical columns like discharge time and total time, ensuring that the synthetic data reflects the variability observed in our real benchmark dataset. Third, synthesizer was trained on real battery data. This process involved teaching the synthesizer to understand the underlying statistical patterns and dependencies present in the data. Using the trained synthesizer, we generated a synthetic dataset containing 100,000 rows. This synthetic data is a simulated representation of battery performance based on the statistical patterns learned from our real dataset.

STRUCTURED RESEARCH OVERVIEW

Problem solved and why it matters

This supplement maps the supervisor's requested structure to evidence already present in the submitted thesis and the original project notebook. It does not replace the manuscript and introduces no new experiment or fabricated result.

Research problem

The thesis addresses the difficulty of predicting the remaining useful life (RUL) of lithium-ion batteries when battery behaviour changes with ageing and the available operational data must support dependable forecasting. The practical problem is to estimate how many usable cycles remain before end of life while preserving physically meaningful degradation behaviour.

Why the problem is relevant

The introduction links battery degradation to safety, reliability, maintenance, and dependable power supply. It discusses applications in vehicles, aerospace, electronics, and energy systems, and explains that inaccurate assessment can contribute to service interruption or unsafe operation. RUL prediction is therefore relevant to preventive maintenance, replacement planning, and safer battery management.

Evidence already present

Requirement	Existing evidence	Original location
Problem	Accurate online RUL forecasting under ageing-dependent behaviour	Abstract; Chapter 1, pp. 3-4
Relevance	Safety, dependability, failure prevention, and maintenance planning	Chapter 1, pp. 3-4
Research contribution	Synthetic data generation, integrity controls, and model comparison	Section 1.3, p. 6

STRUCTURED RESEARCH OVERVIEW

Suggested solution and method rationale

Suggested solution

The implemented solution combines four connected elements: a real battery-cycle dataset, Gaussian Copula synthetic-data generation, statistical and domain-specific validation, and comparative RUL prediction with LSTM and XGBoost. The thesis reports 100,000 generated rows. The supplied notebook cell currently requests 1,000,000 rows; that configuration mismatch is documented in Annex D and must be reconciled before submission.

Why the method is reasonable

- A data-driven approach avoids constructing a detailed electrochemical model while using measured degradation features directly.
- Gaussian Copula generation learns multivariate statistical dependencies instead of adding arbitrary random noise.
- Correlation matrices, Frobenius norm, sorting, filtering, and monotonic RUL checks provide explicit validation rather than assuming that synthetic data is realistic.
- LSTM is a reasonable candidate because the task is sequential and uses a five-cycle look-back window.
- XGBoost provides a structurally different tree-based regression baseline for comparison with the sequential model.
- RMSE, MAE, R-squared, and adjusted R-squared provide complementary error and goodness-of-fit measures.

Method boundary

The implementation is a research prototype evaluated on one battery dataset. The method should not be described as a production battery-management system or as universally validated outside the studied data.

STRUCTURED RESEARCH OVERVIEW

Implementation and recorded results

Implementation chain

Stage	Implemented action	Evidence
1. Inspect	Load cycle-level data; inspect columns, distributions, and RUL behaviour	Notebook cells 1-2; thesis Section 3.1
2. Identify devices	Assign DeviceID when the RUL sequence resets	Notebook cell 4
3. Generate	Fit GaussianCopulaSynthesizer and sample raw rows (thesis: 100,000; notebook setting: 1,000,000)	Notebook cell 5; Figures 10-11; Annex D
4. Clean	Filter invalid decrement values, group by device/cycle, and enforce decreasing RUL	Notebook cells 6-7; Figures 12-14
5. Validate	Compare correlations and calculate the Frobenius norm	Notebook cell 12; Sections 3.3-3.4
6. Prepare	Select features, scale values, create five-cycle sequences, and split data	Notebook cell 19
7. Model	Train LSTM and XGBoost; calculate regression metrics	Notebook cells 21-28; Section 5.2

Recorded model comparison

Model	RMSE	MAE	R-squared	Adjusted R-squared
XGBoost	87.133	24.760	0.9280	0.9276
LSTM	85.405	15.554	0.9308	0.9307

These values are the outputs stored in the supplied notebook. LSTM has the lower RMSE and MAE and the higher R-squared values in that recorded run. The numerical difference in R-squared is small, so the conclusion should remain limited to this dataset and implementation.

STRUCTURED RESEARCH OVERVIEW

Discussion, conclusion, and authorship boundary

Discussion of the applied method

- The dominant degradation relationship was retained: the thesis reports a correlation of approximately -0.999 between Cycle_Index and RUL in both real and synthetic data.
- Not every relationship transferred equally. The thesis reports a change in the Discharge Time versus RUL relationship, showing that synthetic fidelity is partial rather than exact.
- The recorded LSTM result is stronger than XGBoost on the selected metrics, which is consistent with the sequential nature of the problem, but it does not establish universal superiority.
- The thesis conclusion is best understood as evidence for this dataset, preprocessing chain, train-test split, and recorded model configuration.

Conclusion

The work demonstrates an end-to-end path from real battery-cycle observations to statistically checked synthetic data and comparative RUL prediction. Its practical achievement is the integration of generation, domain cleaning, correlation validation, temporal preparation, and two model families in one reproducible notebook workflow.

Required author review

The original thesis manuscript, project notebook, code, datasets, and recorded experimental outputs are attributed to Nayeem Faisal. These supplemental pages are an editorial organization and reproducibility audit prepared from the supplied artifacts. They add no new experiment. The author must review and approve every supplemental statement and follow the supervisor's and THA's disclosure requirements before submission.

LIST OF FIGURES

Figure 1 The relationship between deep learning and other algorithms (Wang et al., 2021). 13

Figure 2 The relationship between deep learning and other algorithms (Wang et al., 2021). 14

Figure 3 The overall structure of the DCNN prediction process (Wang et al., 2021). 14

Figure 4 The iterative residual RUL prediction process using deep learning strategies (Wang et al., 2021). 17

Figure 5 The architectural flow of the LSTM-based prediction network (Bermudez et al., 2017, Wang et al., 2021) 18

Figure 6 Cyber system: Cloud-based battery modeling (Shi et al., 2023) 19

Figure 7 Moving time window architecture for RUL detection (Gu et al., 2021). 20

Figure 8 The general architecture of CNN (Ciccarelli et al., 2023) 34

Figure 9 Preprocessing Results 46

Figure 10 Initialization of Synthesizer 47

Figure 11 Fitting and Generating the Synthetic Data 48

Figure 12 Filtered Synthetic Data 48

Figure 13 Reducing RUL Values 49

Figure 14 Saving and Loading the Synthetic Data 49

Figure 15 Header information. 55

Figure 16 View of features with respect to other features 56

Figure 17 Count plot 56

Figure 18 The real data in terms of RUL vs cycle. 57

Figure 19 The real data in terms of Total time vs RUL. 57

Figure 20 The real data in terms of Total time vs RUL. 57

Figure 21 The Synthetic data in term of Time vs Cycles. 58

Figure 22 The Synthetic data in terms of RUL vs Cycles. 58

Figure 23 The Synthetic data in terms of Total time vs RUL. 58

Figure 24 Overview of the correlation matrix. 60

Figure 25 Overview of the correlation matrix. 61

Figure 26 Installation of library 61

Figure 27 Code of LSTM 62

Figure 28 Detailed code of LSTM 63

Figure 29 Code for mean square error, mean absolute error and r2 error. 63

Figure 30 Comparison of XGBoost predictions and LSTM predictions 64

Figure 31 Comparison of Performance Metrics 65

LIST OF TABLES

Table 1 Methods and key findings of the Prediction Method Analysis 10

Table 2 Methods and key findings of the deep learning RUL existing research..... 15

Table 3 Methods and key findings of the parameterization existing research 21

Table 4 The prediction effect comparison result for different deep learning algorithms (Wang et al., 2021) 23

Table 5 Real-World Dataset Correlation Matrix 50

Table 6 Synthetic Dataset Correlation Matrix 51

LIST OF ABBREVIATIONS

Acronym	Explanation
RNN	recurrent neural networks
ANN	artificial neural networks
SVR	support vector regression
RVM	relevance vector machine
AR	auto-regression
GPR	gaussian process regression
CNN	convolutional neural network
ELM	extreme learning machine
RFR	random forest regression
auto-encoder	AE

TABLE OF CONTENTS

ABSTRACT ..... **ii**
LIST OF FIGURES ..... **iii**
LIST OF TABLES ..... **v**
LIST OF ABBREVIATIONS ..... **vi**
TABLE OF CONTENTS **1**
1. INTRODUCTION **3**
1.1. Model-based Prediction **4**
1.2. Data Driven Methods **5**
1.3. Contribution **6**
1.4. Outline **6**
2. LITERATURE REVIEW **7**
2.1. Various RUL Prediction Methods Analysis **8**
2.2. Deep learning for RUL prediction **12**
2.3. DCNN-based calculation flowcharts **16**
2.4. Application Analysis from the Literature **17**
2.5. Deep Learning for Parameterization **19**
2.6. Comparison of the RUL Prediction Effect **22**
2.7. Insights and Future Directions **23**
2.8. Battery RUL Prediction and Lifetime Extension Strategy **24**
2.9. General Flow **25**
2.10. Signal Preprocessing before Machine Learning **26**
2.10.1. Empirical Mode Decomposition (EMD) 26
2.10.2. Variational Modal Decomposition (VMD) 27
2.10.3. Box-Cox Transformation 27
2.10.4. Wavelet Decomposition Technology (WDT) 28
2.11. Machine Learning Methods **29**
2.11.1. Recurrent neural network 29
2.11.2. Echo state network: 29
2.11.3. Long short-term memory: 29
2.11.4. Gated recurrent unit: 31
2.11.5. Support vector regression 32
2.11.6. Auto-Regression 33
2.11.7. CNN 34
2.11.8. Extreme learning machine 35
2.11.9. Relevance vector machine 37
2.11.10. Gaussian process regression 38
2.12. Comparison of RUL Prediction Methods **40**
2.13. Insights and Future Directions **40**
3. INTRODUCTION TO DATA **41**
3.1. Real World Dataset **41**
3.2. Synthetic Generation **43**

3.2.1. Importance of Synthetic Data Generation 43
3.2.2. Preprocessing and Assigning Unique Device IDs 46
3.3. Correlation between Synthetic Data with real-world Datasets 49
3.3.1. Correlation Analysis 50
3.3.1.1. Real-World Dataset Correlation Matrix 50
3.3.1.2. Synthetic Dataset Correlation Matrix 51
3.3.2. Frobenius norm 52
3.4. Detailed Analysis 52
3.4.1. Maximum and Minimum Values 52
3.4.2. Standard Deviation Variations 53
3.4.3. Missing Values 53
3.4.4. Key Observations 53

Chapter 4 Exploratory Data Analysis (EDA) 54

4.1. Implementation on Dataset 55
4.2. Feature Representation 55

Chapter 5 Feature Engineering, Machine Learning and Conclusion 60

5.1. Feature Engineering: 60
5.2. Machine Learning: 61
5.2.1. LSTM 61
5.2.2. XGBoost 63
5.3. Conclusion 65

References 67

1. INTRODUCTION

Lithium-ion batteries are extensively utilised due to their manifold benefits, including but not limited to elevated energy density, prolonged cycle life, and environmentally sustainable characteristics. There are numerous industries in which these batteries are utilised, including the aerospace, military, and electronics sectors (Armand et al., 2020). As the depletion of conventional nonrenewable resources escalates and industrial society demands more new energy sources, lithium batteries will become more common in energy systems, electric vehicles, weapons, and equipment. These batteries feature low environmental pollution, a lengthy service life, a low self-discharge rate, and a high energy reserve. As a result, lithium batteries possess a highly probable position as the basis of the prospective energy storage industry and hold a vast market sector.

Despite the numerous benefits that lithium-ion batteries offer, their performance will start to decline as the duration of service and the number of charge-discharge cycles grow. Lithium-ion batteries, which function as energy storage and supply components, frequently lead to system failures and different safety incidents due to their deterioration.

In 2006, the Mars Exploration Satellite, which was launched by NASA, experienced a power supply failure that resulted in the loss of communication with Earth (Howard, 2020). Similarly, in 2013, two Boeing 787 "Dream Liners" aircraft were permanently grounded after a fire triggered by faulty electrical systems. In 2016, Samsung issued an urgent recall of its Galaxy Note 7 phones due to numerous explosions attributed to faulty lithium-ion batteries. During the initial half of 2016, two 787 "Dream Liners" aircraft were permanently grounded (Yu, 2020).

The fire incidents that have ensued due to the widespread use of lithium-ion batteries in various domains have significantly compromised the security of both individuals and their possessions. Consequently, there is an imperative need for solutions that address the concerns regarding the dependability and safety of lithium-ion batteries. Safety evaluations of the efficacy of lithium-ion batteries have thus become an extremely important task due to the growing prevalence of such applications.

The assessment of lithium-ion battery availability now consists primarily of three components: RUL prediction, battery health status evaluation, and battery charge status evaluation. The time needed from the current prediction point to the battery's end of life, which is typically indicated by the charge-discharge cycle period, is what is recognised as the RUL prediction among them. The internal resistance of the batteries gradually rises, and their capacity falls with each charge and discharge.

It is believed that a battery's life will end when its remaining capacity reaches 70%–80% of its initial value. The battery will no longer be used for electricity to ensure safety. RUL is a popular study direction in the field of lithium-ion battery research because it has a highly intuitive and significant value for the safety assessment of lithium-ion batteries (Zhang et al., 2019).

Three types of RUL prediction approaches are currently available for estimating the remaining life of lithium-ion batteries: physical model-based RUL prediction methods, data-driven RUL prediction methods, and model-data fusion-based RUL prediction methods. The remaining life prediction techniques for lithium-ion batteries can now be categorised into three categories: data-and physical model-based prediction (Li et al., 2020).

The objective of this research initiates by optimizing the hyperparameters of LSTM and XGBoost by dealing with the challenge of manually tuning parameters and enhancing the algorithm's capability to escape local optima. Then, we utilized a synthesizer for generating synthetic data is adopted for numerical columns like discharge time and total time, ensuring that the synthetic data reflects the variability observed in our real benchmark dataset. Next, synthesizer was trained on real battery data. This process involved teaching the synthesizer to understand the underlying statistical patterns and dependencies present in the data. The results have shown better results in terms of different evaluation metrics.

1.1. Model-based Prediction

These techniques primarily include creating an equivalent model of a lithium-ion battery, incorporating its operational parameters and failure mechanism throughout its life cycle, and using the equivalent model to forecast the lithium-ion battery's RUL. In order to assess the

degradation of lithium-ion battery capacity, (Wang et al., 2016) developed a state-space model of capacity and implemented a spherical volume particle filter (SCPF) to solve the model and obtain the RUL prediction of lithium-ion batteries.

In their study, (Jiao et al., 2020) introduced a novel PF framework that utilises a re-weighting strategy and a conditional variational autoencoder (CVAE) to forecast the RUL of batteries.

Despite the fact that these methods are capable of generating RUL predictions, the following drawbacks remain: One of the challenges encountered is the adjustment of model parameters. 2) The observer technique, including particle filters, is susceptible to particle impoverishment issues, which can result in RUL predictions that are not entirely accurate.

1.2. Data Driven Methods

Unlike model-based methods, data-driven methods eliminate the need to construct a complex electrochemical model of lithium-ion batteries. The prediction of Remaining Useful Life (RUL) is primarily achieved by extracting implicit information from lithium-ion battery capacity degradation data. Most of these techniques utilize evolutionary algorithms, machine learning algorithms, and neural network algorithms (Oji et al., 2021).

In their study, (Zhang et al., 2022) introduced an innovative online synthesis approach that combines artificial neural networks (ANN) with partial incremental capacity to estimate the RUL and State of Health (SOH) under constant current discharge. To smooth the initial incremental capacity curves, the authors utilized advanced filtering methods. They employed correlation analysis methods to extract strongly correlated feature values from the partial incremental curves. RUL prediction is subsequently implemented via an ANN model.

(Gou et al., 2020) introduced a novel hybrid ensemble data-driven approach to effectively forecast the RUL of lithium-ion batteries. To predict the trajectory of battery degradation, a health indicator was selected as the feature input following a Pearson correlation analysis. Model-based methods are more complicated, time-consuming, and cumbersome compared to data-driven methods.

1.3. Contribution

The main contribution of this research are as follows:

- With the metadata at our disposal, we configured a Gaussian Copula Synthesizer, a powerful tool for generating synthetic data while maintaining statistical dependencies. We specified distributions for numerical columns such as discharge time and total time, ensuring that the synthetic data reflects the variability observed in our real dataset.

- Utilizing the trained synthesizer, we generated a synthetic dataset comprising 100,000 rows. This synthetic data serves as a simulated representation of battery performance, derived from the statistical patterns learned from our real dataset.

- To guarantee the quality and relevance of the synthetic data, we conducted several post-processing steps. These steps involved sorting the data by device ID and Remaining Useful Life (RUL), and filtering out rows with positive decrement values to maintain consistency with our real data.

- The thesis starts by optimizing the hyperparameters of LSTM and, addressing the challenge of manually tuning LSTM parameters and enhancing the algorithm's capability to escape local optima. In addition, we designed hyperparameter tuning for an XGBoost regression model using grid search.

1.4. Outline

The remainder of this thesis is organized as follows: Section 2 presents the literature review on RUL prediction of lithium batteries. Chapter 3 introduces the benchmark datasets of the research. The Exploratory Data Analysis (EDA) of the benchmark datasets are presented in Chapter 4. Finally, Chapter 5 concludes the study with future insight.

2. LITERATURE REVIEW

This section discusses the existing studies conducted by the researchers in the RUL prediction of lithium batteries.

Lithium-ion batteries have found widespread application in various domains, including electric vehicles (EVs) and portable electronic devices, due to their notable improvements in energy density, lifespan, and self-discharging (Sanguesa et al., 2021). However, the complex operating conditions, including temperature variations and different charging-discharging cycles, can accelerate negative changes in the internal electrochemical constituents of lithium-ion batteries. Internal mechanical stresses and external variables including operation temperature, depth of discharge (DOD), charge and discharge current magnitude, and ambient humidity impact these modifications, which include the increased internal resistance and the depletion of active material (LAM) and lithium inventory (LLI) (Xiong et al., 2020). Consequently, the continued useful life (RUL) of a battery must be

precisely predicted in order to guarantee dependable operation, due to the capacity and performance deteriorations that occur.

In addition to reflecting battery life, RUL prediction plays a vital role in battery lifetime extension strategies. Extending battery lifetime is essential to maximize the profitability of applications, optimize battery management systems, and improve overall energy management strategies. Various methods have been investigated for extending battery lifetime, including charging and discharging profile optimization, thermal management optimization, and integration with the power grid (Kumar et al., 2023). Surprisingly, despite its importance, the available literature often neglects the potential of using RUL predictions to extend battery lifetime, which is critical for effective battery application.

To address these challenges, this review article explores the significance of RUL prediction for lithium-ion batteries. In this study, we undertake an analysis and comparison of physics-based and data-driven approaches to predict RUL. Specifically, we examine how data-driven methods rely on historical and operational data, while physics-based methods heavily rely on electrochemical models and equivalent circuit models.

Extensive research has been conducted on machine learning algorithms, including RNN, ANN, SVR, RVM, AR, GPR, CNN, ELM, RFR, and auto-encoders, to achieve precise RUL prediction (Li et al., 2023). Furthermore, the development of the fundamental research components of current machine learning algorithms is monitored by examining relevant research papers from the previous decade.

The objective of this research attempt is to present an enhanced RUL prediction model by employing deep learning algorithms, specifically the extreme learning machine algorithm—high-accuracy deep convolutional neural network. Through a detailed mathematical analysis, we evaluate different modeling methods and strategies, highlighting their key advantages and realizing accurate RUL predictions (Che et al., 2023). Additionally, we analyze the changing process of lifetime characteristics, providing valuable insights into the realization technology of RUL prediction.

The accurate prediction of the RUL for lithium-ion batteries is achieved through the utilization of differential optimization and k-nearest neighbor regression techniques (Zhou et al., 2020). To achieve this, experimental data must be collected for every internally connected cell in the battery pack. In the context of health status surveillance, the implementation of enhanced unscented Kalman filtering (UKF) and the time convolutional network (He et al., 2022) can yield reliable RUL predictions (Wang et al., 2020).

Liu et al. (2021) define F-distributed particle filtering (PF) and kernel averaging (Chen et al., 2020a,b) as the methodologies employed to forecast the RUL of aircraft lithium-ion batteries. (Zhang et al., 2020) To implement nonlinear drift fractional Brownian motion in the prediction process, multiple hidden variables are utilized. Yin et al. (2019) state that comprehensive treatments for distance anxiety also incorporate the prediction and cognition of degradation dynamics.

Additionally, it is possible to optimize the cycle life evaluation of various formulations using the hybrid prediction method (Ma et al., 2020a). To achieve an accurate RUL prediction, multi-parameter optimization is performed (Zhou et al., 2020a). This optimization is carried out utilizing the electrochemical model as the battery framework (Farmann and Sauer, 2018). Liu et al. (2020a) employ various modeling techniques to assess the effects of battery calendar aging prediction. Among these, time series modeling is utilized to forecast the RUL value. By combining the mutated long-short-term memory (LSTM) algorithm and the neural network (NN) model (Liu et al., 2020a) with recurrent neural network (RNN) and multiple linear regression (Zhao et al., 2019a) techniques, it is possible to achieve precise RUL prediction. This approach effectively captures degradation information.

The use of indirect health indicators and Gaussian process regression can also be employed to predict battery life (Pan et al., 2020). Sun et al. (2021) also suggest employing the rank correlation coefficient prediction method as a means to acquire the RUL value of lithium-ion batteries. The development of the RUL prediction model takes into account the characteristic of idle time (Cui et al., 2020). Additionally, Jiang et al. (2019) propose an enhanced method for predicting the decline in capacity via odorless particle filtration.

EVIDENCE MAP

Table 1. Prediction-method literature comparison

The reviewed studies use statistical filtering, state observers, Gaussian-process methods, physics-based models, and neural networks to estimate battery health, remaining useful life (RUL), or cycle life. The table standardizes the wording so that each row states the cited study, its principal method, its research focus, and the reported contribution without implying that results from different datasets are directly comparable.

Study	Method / model	Research focus	Principal reported finding
Zhou et al. (2020a)	K-nearest-neighbor regression with differential optimization	RUL estimation from experimental lithium-ion cell data	Reported accurate RUL estimates across internally connected cells.
Lin et al. (2020a)	Time-convolutional network	Battery health-state monitoring	Reported robust RUL prediction.
Wang et al. (2020a,b,c)	Enhanced unscented Kalman filtering	Battery health-state monitoring	Reported robust RUL estimation under uncertain measurements.
Zhu et al. (2019)	Comparative RUL prediction methods	Lithium-ion batteries with multiple uncertainties	Compared prediction behavior and evaluation criteria under uncertainty.
Zhao et al. (2019a); Zhang et al. (2019b)	Nonlinear prediction with recovery-effect modeling	RUL prediction with capacity-recovery effects	Estimated RUL while accounting for recovery phenomena.
Liu et al. (2021)	F-distributed particle filter with kernel smoothing	Aircraft lithium-ion batteries	Combined particle filtering and kernel smoothing for RUL estimation.
Zhang et al. (2019a-e)	Nonlinear-drift fractional Brownian motion with hidden variables	RUL under nonlinear degradation	Modeled nonlinear drift by incorporating multiple latent variables.
Zheng et al. (2020)	Optimal time-series health index	RUL prediction	Used an optimized time-series health index for RUL estimation.
Li et al. (2021a-c)	Adaptive UKF with optimized support-vector regression	RUL prediction	Combined adaptive state estimation with SVR-based prediction.
Xiong et al. (2020a,b)	Improved particle filter, Wiener process, and unscented particle filtering	RUL prediction	Combined improved filtering with stochastic degradation modeling.

EVIDENCE MAP

Table 1. Prediction-method literature comparison (continued)

Standardized terminology; study-specific meaning retained from the cited source and bibliography.

Study	Method / model	Research focus	Principal reported finding
Tian et al. (2020)	Fractional Brownian-motion model	RUL prediction	Modeled RUL using fractional Brownian-motion degradation behavior.
Yin et al. (2019)	User-specific driving patterns and capacity-recovery effects	Capacity-degradation dynamics	Combined driving behavior and recovery effects to predict degradation.
Qiu et al. (2020a)	Improved cuckoo-search particle filter	RUL prediction	Fused failure-time and field-degradation information through random effects.
Yang et al. (2019)	Adaptive data-driven feature-enhancement optimization	Accurate RUL prediction	Proposed an optimized feature-enhancement method for RUL estimation.
Farmann and Sauer (2018)	Electrochemical model	Model-based RUL prediction	Used an electrochemical model as the battery-degradation framework.
Liu et al. (2020a)	Mutated LSTM and neural-network models	RUL prediction	Reported high-accuracy RUL estimation using neural-network approaches.
Zhao et al. (2019b)	Takagi-Sugeno observer	Fault-tolerant lithium-ion battery systems	Designed an observer that adapts to fault conditions.
Pan et al. (2020); Hu et al. (2020)	Gaussian-process regression with indirect health indicators	Battery-life prediction	Combined indirect health indicators with Gaussian-process regression.
Sun et al. (2021)	Rank-correlation-coefficient prediction method	RUL prediction	Constructed an RUL model using ranked feature relationships.
Cui et al. (2020)	Rest-time characteristic	RUL prediction	Included resting-time behavior in the RUL model.

EVIDENCE MAP

Table 1. Prediction-method literature comparison (continued)

Standardized terminology; study-specific meaning retained from the cited source and bibliography.

Study	Method / model	Research focus	Principal reported finding
Jiang et al. (2019)	Copula-based battery-pack consistency model	Pack consistency and energy-utilization efficiency	Modeled battery-pack consistency for energy-utilization-efficiency estimation.
Li et al. (2018)	Second-order central particle-filter algorithm	RUL prediction	Reported RUL estimation using a second-order central particle filter.
Tian et al. (2019)	Gray model with a score-processed indicator	RUL prediction	Constructed a prediction framework around a processed health indicator.
Tang et al. (2019a)	Gaussian-process regression with a weighted-average operator	RUL prediction	Introduced a fusion-based selection and aggregation method.
Severson et al. (2019)	Early-cycle data-driven features	Cycle-life prediction before capacity degradation	Predicted battery cycle life before pronounced capacity degradation.
Liu et al. (2019)	Modified Gaussian-process regression	Cycle-capacity prediction	Analyzed cycle-capacity prediction using modified GPR models.
Cong et al. (2020)	Health indicators with Gaussian-process regression	Cycle-capacity prediction	Predicted cycle capacity from health indicators.
Xiong et al. (2018)	LSTM and Elman neural-network models	Cycle-life prediction	Used hybrid neural modeling for cycle-life prediction.
Jiang et al. (2018)	Low-temperature internal-heating strategy	Battery-pack thermal operation and lifetime preservation	Proposed internal heating without reducing battery lifetime.
Eleftheroglou et al. (2019)	Intelligent data-driven prognostic methods	Real-time RUL to end-of-discharge for UAV lithium-polymer batteries	Estimated end-of-discharge RUL with uncertainty quantification.

Editorial corrections. The Jiang et al. (2018) and Jiang et al. (2019) rows now follow the titles listed in the thesis bibliography. The Severson et al. (2019) row describes its early-cycle data-driven prediction contribution, and the Eleftheroglou et al. (2019) row now uses the correct term end-of-discharge rather than 'UAV emission'.

2.2 Deep learning for RUL prediction

Deep-learning methods can learn nonlinear relationships between operating measurements, degradation indicators, and battery life. Feedforward networks, extreme-learning machines, recurrent networks, and LSTM models differ in how they represent temporal information. Figure 1 positions deep learning within the wider family of reviewed RUL methods; the following sections discuss the architectures and their reported battery-prognostic applications.

Deep learning is utilized to perform the data-driven prediction; specifically, a deep neural network (DNN) is employed to forecast the battery RUL value through the implementation of an initial data-driven process. SVM, k-NN, ANN, and LR are all components of the ML algorithm (Hafiz et al., 2020).

For the purpose of battery RUL prediction, an iterative DNN structure (Zhou et al., 2020b) is developed. This structure consists of two LSTM layers, LSTM1 and LSTM2, which are utilized to extract features. The battery parameters remain constant across all layers (Zhang et al., 2020), thereby enabling this architecture to effectively accommodate varying network input quantities. Figure 2 illustrates the comprehensive architecture of the deep learning prediction model.

A performance enhancement can be achieved by adjusting the structure of the prediction model to correspond with the immediate verification result of the measurement data. Following the feature extraction process of two LSTM layers, the final LSTM2 unit's output, which comprises the information extracted from the entire input matrix, can be utilized for additional RUL prediction. Following that, neurons comprise a full-connected layer atop the future prediction network. Consequently, the LSTM unit's structure is modified.

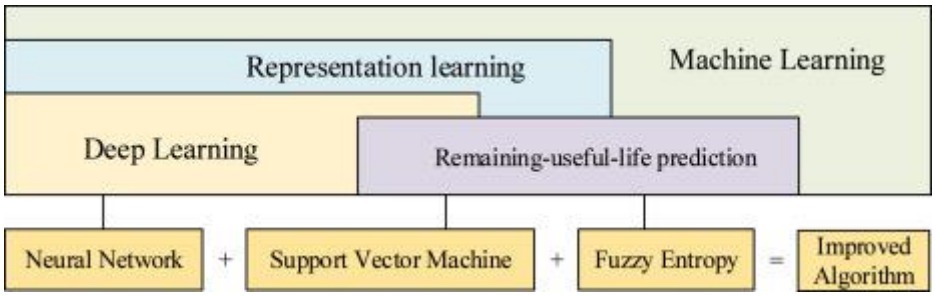


Figure 1 The relationship between deep learning and other algorithms (Wang et al., 2021).

The health degradation patterns of lithium-ion batteries are predicted utilizing deep Gaussian regression (Tagade et al., 2020a). By integrating and transfer learning of lithium-ion batteries, an enhanced DCNN model is constructed (Shen et al., 2020a).

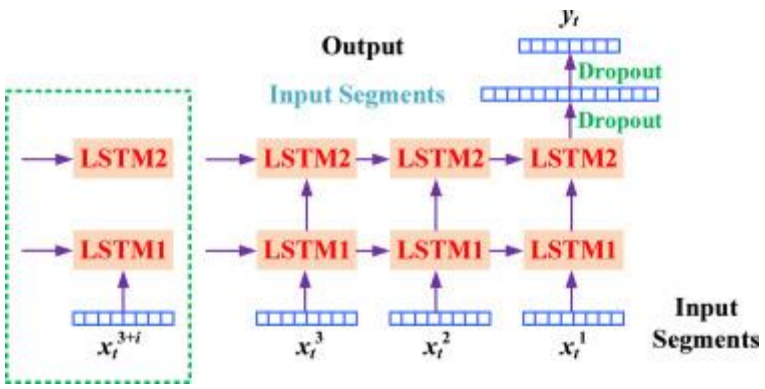


Figure 2 The relationship between deep learning and other algorithms (Wang et al., 2021).

The authors utilize 10-year daily-cycle data from 8 implantable lithium-ion battery cells as the data source set (Lin et al., 2020). These data points are utilized to pre-train 8 DCNN sub-models. Figure 3 illustrates the DCNN prediction process in its general framework.

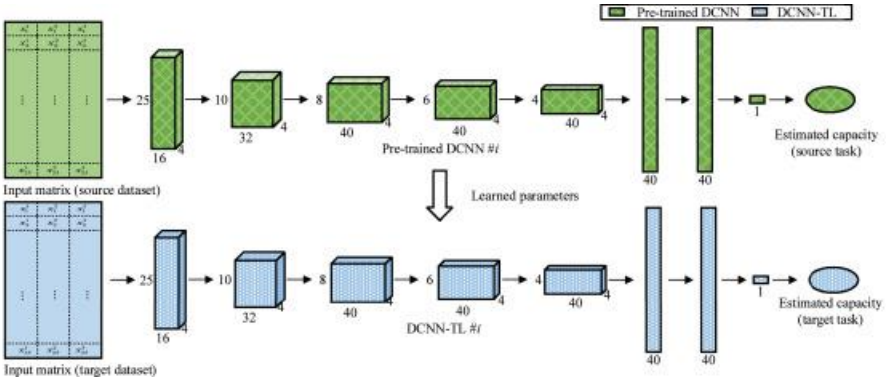


Figure 3 The overall structure of the DCNN prediction process (Wang et al., 2021).

Figure 3 illustrates the application of five data-driven DCNN-TL submodels and comprehensive calculation strategies to achieve the RUL prediction (Ng et al., 2020).

The effective realization of battery characterization and RUL prediction can be achieved by applying terminal voltage and current iteratively. The authors derive the changing laws of each parameter from experimental tests conducted under different working conditions (Zhang et al., 2019b). Additionally, Mejdoubi et al. (2019) propose an enhanced framework for predicting RULs by integrating the mathematical state-space description and model

parameter identification. Time-varying parameters are vulnerable to the influence of a range of environmental factors, including temperature conditions.

Table 2 Methods and key findings of the deep learning RUL existing research

Authors	Methods	Focus Area	Key Findings
Guha and Patra (2018a)	Extreme Learning Machine (ELM), Feedforward Neural Network (FNN)	Cycle life prediction for lithium-ion batteries	ELM has fast learning speed and effective generalization compared to FNN
Ma et al. (2020a)	ELM, Generalized Learning (BL)	Cycle life prediction for lithium-ion batteries	Introduction of BL idea, improved generalization performance
Du et al. (2019)	Improved ELM, Data-driven non-parametric aging analysis	Storage and cyclic operation prediction	Realizes storage and cyclic operation from operating data
Li et al. (2020a,b,c,d)	Improved Gaussian regression, Nonlinear regression	Battery health status prediction	Construction of multi-time scale framework for predicting battery health status
Chehade and Hussein (2020)	Cooperative Gaussian processing regression	Transfer learning of capacity trends	Establishment of model for capacity trends transfer learning between battery cells
Wei et al. (2020)	Improved deep learning, Deep convolutional neural network (DCNN)	Online capacity changes prediction	DCNN estimates battery power portion based on voltage, current, and temperature measurement
Meng et al. (2021)	Hybrid wavelet-adversarial deep model	Solar power generation prediction	Accurate prediction of solar power generation
Zou et al. (2021)	Reinforcement learning, Unscented transform	Stochastic multi-carrier energy management	Management of energy in smart islands
Lan et al. (2021)	Advanced ML model, Consideration of HEV charging demand	Energy management of renewable microgrids	Efficient energy management of microgrids considering HEV charging demand
Liu et al. (2020)	Improved RetinaNet	Defect recognition of transmission lines	High-precision defect recognition method
Hafiz et al. (2020); Wu et al. (2019); Shin et al. (2019); Eleftheroglou et al. (2019b); Lipu et al. (2018)	ML algorithms (SVM, k-NN, ANN, LR)	Degradative mode recognition of lithium-ion batteries	Extraction of degradative mode using impedance spectrum
Ma et al. (2021b)	Deep learning, RNN, LSTM	Adaptive time-series prediction and online verification	Combination of effective variables for adaptive time-series prediction and online verification
Zhou et al. (2020b)	Iterative DNN structure, LSTM layers	Battery RUL prediction	Feature extraction using LSTM layers for scalable network input sizes

Tagade et al. (2020a)	Deep Gaussian regression	Health degradation patterns prediction	Prediction of health degradation patterns
Shen et al. (2020a)	Improved DCNN model, Integrated and transfer learning	Lithium-ion batteries prediction	Pre-training on 10-year daily-cycle data for accurate RUL prediction
Lin et al. (2020)	Data-driven DCNN-TL submodels	RUL prediction	Utilization of DCNN submodels for iterative calculation and accurate RUL prediction
Ng et al. (2020)	Equivalent circuit model (ECM), Distributed feature modeling	Dynamic battery characterization	Accurate dynamic battery characterization using ECM and state-space equation
Jiao et al. (2020); Ma et al. (2021a)	Equivalent modeling, Dynamic battery characterization	RUL prediction	Establishment of equivalent model for full-life-cycle battery working conditions
Corno and Pozzato (2020)	Electrochemistry, ECM	Temperature impact analysis	Analysis of temperature impact on model structure
Masaud and El-Saadany (2020)	Experimental tests, Changing law analysis	Battery characteristics simulation	Simulation of battery characteristics through changing law analysis
Mejdoubi et al. (2019)	Mathematical state-space description, Model parameter identification	RUL prediction	Improved framework for RUL prediction combining mathematical state-space and parameter identification

2.3. DCNN-based calculation flowcharts

The objective of the DCNN processing is to gather data samples and modify the weight-position coefficients of the particles. The corrected empirical distribution is then determined using the measured results (Yang et al., 2020). While maintaining the same level of accuracy in the RUL prediction, it is possible to decrease the quantity of particle samples, consequently resulting in a reduction in the calculation volume. As shown in Equation (2), the learning transfer into knowledge learning is performed (Shen et al., 2020b) using the data source set encoded as the parameters of n single DCNN sub-models.

$$\theta^S = \{\theta_1^S, \theta_2^S, \dots, \theta_n^S\} \quad (2)$$

The entire DCNN modeling particle-matrix εS structure can be represented by n individual DCNN sub-modeling factors of δn^S, where n denotes the quantity of individual DCNN sub-models, as shown in Equation (2).

The purpose of DCNN processing is to gather data samples and modify the weight-position coefficients of the particles. The corrected empirical distribution is then derived from the measured results (Yan).By utilizing the data set from the source task, the pre-trained DCNN model is modified via transfer learning (Xu et al., 2018). In order to validate the performance of the model in the intended task, short-term cyclic experimental tests are conducted till the long-term cyclic experimental test becomes feasible (Hong et al., 2020a). By employing this method, it is possible to transform knowledge efficiently with a limited data set, resulting in decreased demands for data acquisition while maintaining consistent accuracy in predictions (Finegan et al., 2020).

It is feasible to reduce the number of particle samples utilized for the RUL prediction while maintaining its accuracy, which would subsequently lead to a decrease in the computation volume. As illustrated in Equation (2) (Shen et al., 2020b), the process of transforming learning into knowledge learning entails encoding the parameters of n single DCNN sub-models from the data source set.

Consequently, the DCNN-ETL algorithm can be utilized to construct the entire structure (Wei et al., 2018). The transfer learning workflow is illustrated in Figure 4.

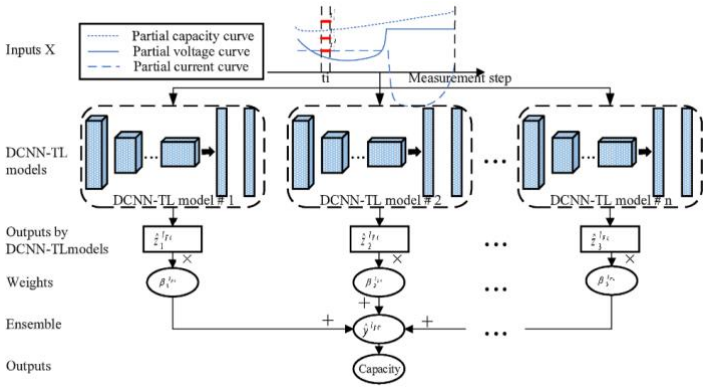


Figure 4 The iterative residual RUL prediction process using deep learning strategies (Wang et al., 2021).

2.4. Application Analysis from the Literature

The neural network, which can adapt to the pulse current operating conditions, is developed to forecast the aging trajectory of batteries (Tang et al., 2020). Empirical mode

decomposition can also be employed in conjunction with the ANN to forecast the RUL value of lithium-ion batteries (Wu et al., 2020).

The improved extreme learning machine (ELM) algorithm is utilized to generate the RUL prediction (Liu et al., 2020a). Utilizing the comprehensive ELM method, capacity and cycle life forecasts for lithium-ion batteries are also accomplished (Ma et al., 2020a). Zhao et al. (2020) propose a method for predicting life by combining the attention mechanism and fusion model.

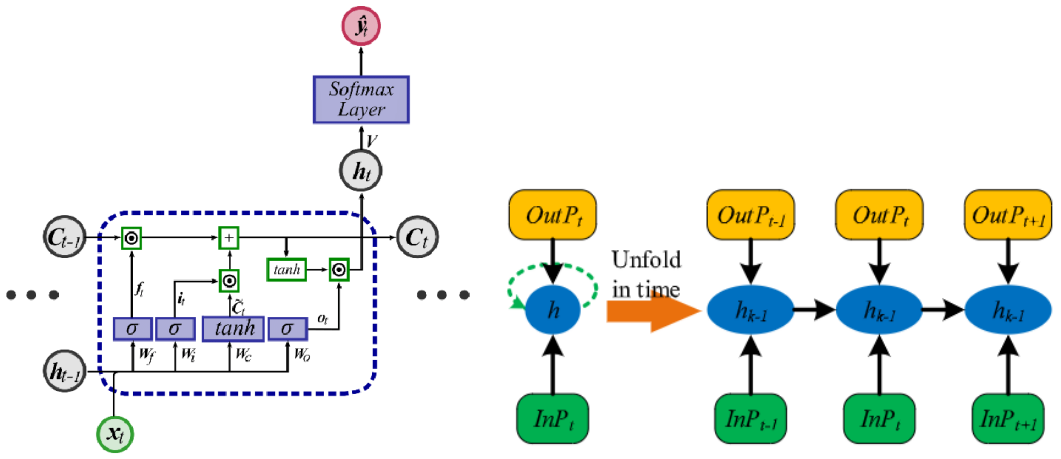


Figure 5 The architectural flow of the LSTM-based prediction network (Bermudez et al., 2017, Wang et al., 2021)

Li et al. (2019) demonstrated the utilization of a CNN to reliably determine the parameters of an electrochemical lithium-ion battery model. An integrated data mining framework is developed by Seruga et al. (2021) for the expression and prediction of battery characteristics, and a novel machine learning approach is suggested to achieve battery health management (Shen et al., 2021).

Deep learning algorithms efficiently automate the process of feature extraction with the least amount of domain expertise and human labor (Hong et al., 2020b). By implementing these algorithms in a hierarchical data representation architecture, it is possible to extract sophisticated functions from the final network layer (Lui et al., 2021b).

Therefore, the RUL prediction process incorporates the deep learning algorithm, illustrated in Figure 6 as its comprehensive framework.

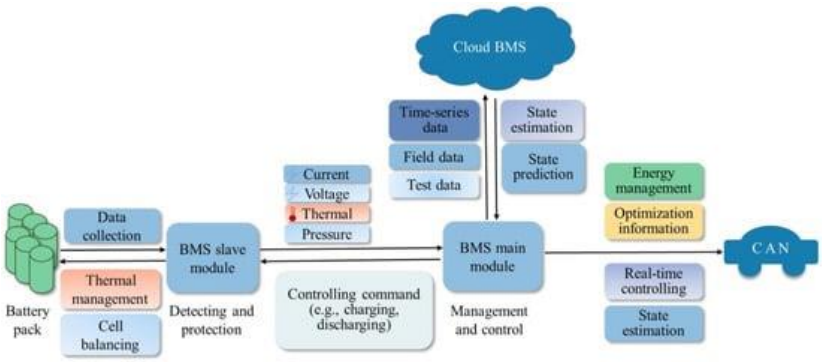


Figure 6 Cyber system: Cloud-based battery modeling (Shi et al., 2023)

2.5. Deep Learning for Parameterization

A deep recursive kernel function and Gaussian processing regression are utilized to predict the RUL for lithium-ion batteries (Ma et al., 2020c). To address the long-term dependence, a gated recursive unit (GRU) is introduced (Ma et al., 2020b). Furthermore, the prediction and verification of lithium-ion battery capacity is achieved through the implementation of LSTM and RNN algorithms (Liu et al., 2020b).

For estimating the RUL value of lithium-ion batteries, the DNN and DCNN algorithms are also implemented (Hong et al., 2020a). In addition, the network incorporates the packet loss technique by 0.5, which is a highly effective regularization method (Chu et al., 2020), in order to prevent the over-fitting phenomenon of the training data. Relied linear unit (ReLU) activated functions are utilized by the full-connected layer to prevent gradient diffusion or disappearance (Atalay et al., 2020).

The time window in Figure 7 has the dimensions nt − ns, where nt represents the number of data segments generated by the test samples at time point t.

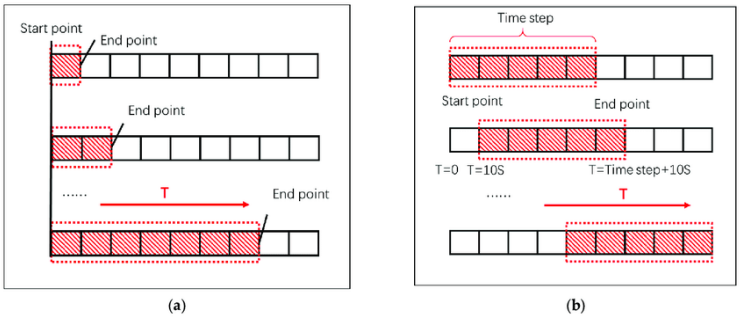


Figure 7 Moving time window architecture for RUL detection (Gu et al., 2021).

Liu et al. (2019b) describe the implementation of dual deep learning coordinated control for the hybrid ESS in the microgrid. The challenging response problem associated with energy management in discrete battery systems can be solved through the implementation of multi-agent deep reinforcement learning (Lu et al., 2020).

Real-time stochastic energy storage system (ESS) optimization is carried out via residential photovoltaic power generation (RPV), enhanced battery energy storage (BES), and DP-based prediction (Kwak et al., 2021). A framework for multi-agent reinforcement learning is developed in order to address the scheduling problem related to lithium-ion batteries (Zhang et al., 2020). Yang et al. (2020c) describe the transfer of energy management strategies to hybrid tracked vehicles, which are predicated on DRL modeling.

The LSTM network, which is employed to forecast various parameters of the EV battery system (Hong et al., 2019), incorporates a pre-loss technology that has been developed to mitigate the issue of overfitting. Utilizing a deep-learning predictor (Afrasiabi et al., 2019), which incorporates a multi-agent framework for early dispatch, microgrid energy management is achieved.

The energy management strategies for electric vehicles are inferred using Bayesian optimization and deep Q-learning (Qiu et al., 2020a). The LSTM algorithm has been demonstrated to accurately predict faults in the battery system (Hong et al., 2019). Deep reinforcement learning is employed to manage the energy consumption of hybrid buses using a continuous state-space expression (Tan et al., 2019). Conversely, the CNN strategy is utilized to identify blister defects (Yang et al., 2020).

In their study on the demanding response, Vazquez-Canteli and Nagy (2019) undertake an examination of reinforcement learning. They specifically focus on its application in coordinating the activities of multi-agent systems that can participate in the demanding response procedures.

Table 3 Methods and key findings of the parameterization existing research

Reference	Techniques	Application Domain	Principal Insights
Ma et al. (2020c)	Gaussian processing regression, Deep recursive kernel function, GRU	RUL prediction for lithium-ion batteries	GRU introduced to handle long-term dependence for RUL prediction
Liu et al. (2020b)	LSTM, RNN	Battery capacity prediction and verification	Utilization of LSTM and RNN algorithms for capacity prediction
Yang et al. (2020b)	Ind-RNN model	Health status evaluation under variable load conditions	Evaluation of health status considering variable load conditions
Lin et al. (2020b)	Energy-saving strategies, Traffic information, EV path planning	RUL prediction and energy management	Incorporation of energy-saving strategies for RUL prediction and EV path planning
Li et al. (2020a)	RNN with convolutional recursive unit	RUL prediction	Introduction of convolutional recursive unit for improved RUL prediction
Xue et al. (2020a,b)	Deep learning, Comprehensive neuropsychological evaluation data	High-precision diagnosis framework	Construction of high-precision diagnosis framework using deep learning
Hong et al. (2020a)	DNN, DCNN	RUL estimation	Estimation of RUL using deep neural network models
Chu et al. (2020)	Packet loss technique	Regularization method	Avoidance of over-fitting in training data through effective regularization
Atalay et al. (2020)	ReLU activation functions	Gradient disappearance or diffusion	Use of ReLU activation to avoid gradient issues
Liu et al. (2019b)	Dual deep learning coordinated control	Microgrid energy management	Coordinated control of hybrid energy storage systems
Lu et al. (2020)	Multi-agent deep reinforcement learning	Energy management of discrete battery systems	Overcoming demand response problem using multi-agent deep reinforcement learning
Ma et al. (2020c)	Deep learning network, Optimal peak drift connection	Household load management	Optimal peak drift connection to the public grid using deep learning
Hong et al. (2020b)	Deep learning, Adaptive dynamic programming	Energy storage method	Proposal of optimized energy storage method using adaptive dynamic programming
Zhang et al. (2018b)	AI and ML algorithms	Target ESS solution	Application of AI and ML to energy storage system solutions

Li et al. (2020a,b,c,d,e,f,g)	Multi-objective deep reinforcement learning	Reliable power dispatch for emission-free ships	Constrained charging using multi-objective deep reinforcement learning
Kwak et al. (2021)	DP-based prediction, Residential photovoltaic power generation (RPV), Improved BES	Real-time stochastic ESS optimization	Optimization of energy storage system considering residential photovoltaic power generation
Zhang et al. (2020a,b,c,d)	Multi-agent reinforcement learning framework	Lithium-ion battery scheduling	Construction of framework for scheduling lithium-ion batteries
Yang et al. (2020c)	Deep reinforcement learning (DRL), Transfer learning (TL)	Energy management for hybrid tracked vehicles	Transfer of energy management strategies to hybrid tracked vehicles
Eleftheroglou et al. (2019b)	DRL, TL, EMS training, Deep deterministic gradient algorithm	Energy management	Execution of deep deterministic gradient algorithm for energy management
Han et al. (2019)	Dual deep Q learning, Reinforcement learning	Energy management for hybrid EVs	Construction of dual learning structure for energy management
Hong et al. (2019)	LSTM, Pre-loss technology	Fault prediction of battery systems	Introduction of pre-loss technology to prevent overfitting in fault prediction
Afrasiabi et al. (2019)	Deep-learning predictor, Multi-agent framework	Microgrid energy management	Realization of energy management using a deep-learning predictor
Qiu et al. (2020a)	Deep Q-learning, Bayesian optimization	EV energy management	Extraction of energy management strategies using combined Q-learning and Bayesian optimization
Tan et al. (2019)	Deep reinforcement learning, CNN strategy	Energy management for hybrid buses, Defect detection	Realization of energy management and defect detection using deep reinforcement learning and CNN
Vazquez-Canteli and Nagy (2019)	Reinforcement learning review	Demand response coordination	Review of reinforcement learning for coordinating multi-agent systems in demand response
Feng and Dong (2020)	Observing point prediction, Probability density function	Mathematical function for time point prediction	Prediction of future time points using observed RUL values and probability density functions
Wang et al. (2019)	Recursive updating, Posterior filtering	Probability density function updating	Iterative calculation and updating process for probability density functions

2.6. Comparison of the RUL Prediction Effect

In this part, the critical criteria are introduced to compare different models in the literature, intending to find the best method for the battery RUL prediction. Then, the research direction of the battery life prediction is discussed. The modeling performance and results of different prediction methods are compared, and the internal structure is also related to the battery life prediction. The parameterized model is realized by changing the parameters, which are

regarded as functional formal constants. As for the online learning process, it may limit the adaptability of the prediction process. Therefore, the accurate prediction ability depends on the reliability of the initial model. Since the ANN model also has a fixed number of weighting coefficients, it has been regarded as a parametric modeling type (Tang et al., 2019). And then, various prediction effects can be displayed in different deep-learning algorithms. The prediction results and effects are obtained and compared by taking RMSE, MaxE, Speed, and Accuracy as evaluation criteria, as shown in Table 4.

Table 4 The prediction effect comparison result for different deep learning algorithms (Wang et al., 2021)

The prediction effect comparison result for different deep learning algorithms.

Algorithms	RMSE (%)	MaxE (%)	Speed	Accuracy
Extreme Learning Machine (ELM)	/	/	/	94.20% (Zhang et al., 2018a)
Deep Convolutional Neural Network (DCNN)	1.986 (Shen et al., 2020a)	6.838 (Shen et al., 2020a)	14.138 for Trial 1 (Shen et al., 2020a)	/
Deep Neural Network (DNN)	1.59 (Cong et al., 2020)	7.14 (Cong et al., 2020)	/	/
Recurrent Neural Network (RNN)	6.87	37.92	/	/
Long Short-Term Memory (LSTM)	MRE < 5% (Hong et al., 2019a,b), RMSE=0.25425 and NRMSE=0.1424 (Afrasiabi et al., 2019)	MAPE=14.785 (Afrasiabi et al., 2019)	/	/
DCNN - Ensemble Transfer Learning (DCNN-ETL)	1.114 (Shen et al., 2020a)	5.487 (Shen et al., 2020a)	133.924 for Trial 1 (Shen et al., 2020a)	/
DCNN with Ensemble Learning (DCNN-EL)	3.539 (Shen et al., 2020a)	16.798 (Shen et al., 2020a)	144.583 for Trial 1 (Shen et al., 2020a)	/
DCNN with Transfer Learning (DCNN-TL)	1.361 (Shen et al., 2020a)	2.452 (Shen et al., 2020a)	12.294 for Trial 1 (Shen et al., 2020a)	/
Adaptive LSTM (ALSTM)	/	/	8 min 46.847 s (Liu et al., 2017b)	93.12% (Liu et al., 2017b)
ALSTM with an attention mechanism	/	/	9 min 26.701 s (Zhang et al., 2017)	97.22% (Zhang et al., 2017)
Average Euclidean distance - Stacked Denoising Autoencoder (AED-SDA)	/	/	/	92.40% for A1 (Ma et al., 2020a)
Gated Recurrent Unit kernel - Gaussian Process Regression (GRU-GPR)	0.79 (Lipu et al., 2018)	7.92 (Lipu et al., 2018)	/	/

In Table 4, it can be known from the comparative analysis that the DP method is very effective for the RUL prediction of lithium-ion batteries. Among them, the DCNN and LSTM strategies are mainly used for accurate RUL prediction. By comparing the parameter influence on the prediction results of the literature, the maximum likelihood method is used to evaluate the prediction results, the implementation process of which is effective when performing the random sampling. Therefore, it is necessary to use a random variable that follows the corresponding distribution type, which can be then introduced into the DCNN-ETL to realize the accurate RUL prediction that is being verified by experimental tests n.

2.7. Insights and Future Directions

The accurate remaining useful life prediction is the key point of lithium-ion battery condition monitoring with high difficulty, and the deep learning method improves the prediction accuracy and robustness.

A specific criterion is defined to evaluate the accuracy and computational cost of the different modeling types. The experimental results show that the deep learning algorithm has non-parametric characteristics, probabilistic capabilities, and powerful computational advantages, which are suitable for the accurate prediction effect. The constructed mathematical functions are particularly convenient for the accurate remaining useful life prediction purposes, providing an effective solution choice. The latest development of modeling methods requires further research and development. By providing a convenient method to develop an adaptive prediction model, the latest technology is proposed, compared, and analyzed. The tentative and adaptive prediction strategies based on deep learning can improve the promotion and application of lithium-ion batteries. By comparing various improved deep learning methods, the high-accuracy deep convolutional neural network—extreme learning machine algorithm will be realized and verified with the near future re- search to realize the accurate remaining useful life prediction by shortening test time, increasing the prediction accuracy, and improving modeling adaptability.

2.8. Battery RUL Prediction and Lifetime Extension Strategy

This part deeply explores the development of RUL prediction using machine learning (ML) algorithms by analyzing relevant papers over the past decade and identifying the ten most frequently used ML algorithms. It also investigates the possibility of extending battery lifetime through RUL prediction results. The study introduces the general flow of RUL prediction and four primary signal pre-processing techniques. A comprehensive comparison of ML algorithms is presented, highlighting improvement opportunities in early prediction, local regeneration modeling, physical information fusion, generalized transfer learning, and hardware implementation. The feasibility of using RUL as an indicator for extending battery lifetime is also discussed. Optimizing the charging profile based on accurate RUL predictions may offer battery lifetime extensions in the future. This section aims to inspire

future advancements in ML algorithms for battery RUL prediction and lifetime extension strategies.

2.9. General Flow

The overall structure of ML-based remaining useful life (RUL) prediction consists of three steps, as illustrated in Figure 3. In the initial step, data is collected from battery aging tests or real-world operations to extract relevant features. These aging features can be categorized as direct or indirect. Common direct features include temperature, charging or discharging current, and output voltage. Indirect features encompass metrics like time intervals of an equal charge voltage difference (TIECVD), incremental capacity analysis (ICA), differential voltage analysis (DVA), and differential thermal voltammetry (DTV). Extracting features from electrochemical impedance spectroscopy (EIS) data has gained popularity, and three main approaches are employed: extracting features from all frequency points, extracting features from equivalent circuit models (after obtaining model parameters from EIS), and extracting features from the fixed-frequency impedance. Another effective method involves using ML techniques such as auto-encoders (AE) to extract features from EIS data. It is crucial to consider temperature, state of charge (SOC), and relaxation effects when analyzing EIS data for RUL prediction. Researchers have quantified the impact of temperature and SOC on the accuracy of state of health (SOH) estimation, highlighting the need to incorporate these factors. Correlation analysis is commonly used to filter out irrelevant features, and signal pre-processing techniques may be employed to smoothen capacity or state of health (SOH) curves or enhance the linear relationship between capacity and features.

The second step involves training ML models to establish the relationship between the extracted features and battery RUL. Alternatively, the ML model can first estimate SOH and then recursively predict the next SOH by using the estimated SOH value in a sliding window until a predefined threshold is reached. Most algorithms used for SOH estimation can also be employed for RUL prediction, with the main distinction lying in the input variables. SOH estimation typically utilizes features directly related to SOH, while RUL prediction often employs the previous SOH or capacity values estimated from the features. The first approach mentioned requires a significant amount of real RUL data from experiments and requires skillful feature extraction, limiting its application conditions. An example of this approach

is seen in a study that employed discharge voltage curves from early cycles to predict cycle life in fast-charging lithium iron phosphate/graphite batteries.

Finally, the performance of the model is evaluated using various metrics such as absolute error (AE), mean absolute percentage error (MAPE), mean square error (MSE), root mean square error (RMSE), mean absolute error (MAE), and max absolute error (MaxAE) based on the test dataset.

In summary, ML-based RUL prediction involves data collection and feature extraction, training ML models to establish the relationship between features and RUL, and evaluating the model's performance using different metrics. The extraction of features from various sources, particularly EIS data, has gained prominence. The consideration of temperature, SOC, and relaxation effect is essential. Different approaches exist, including direct RUL prediction and recursive estimation based on SOH. The choice of approach depends on available data and application conditions.

2.10. Signal Preprocessing before Machine Learning

Regarding battery RUL prediction, the primary focus of most researchers has been on understanding the degradation pattern of the battery. Identifying the RUL point on a smooth capacity or SOH curve is relatively straightforward compared to dealing with noisy and fluctuating curves. Signal pre-processing techniques can effectively separate the main trend curve from the noise and fluctuations. Additionally, the features extracted for RUL prediction often exhibit a linear relationship with the capacity or SOH. Enhancing this linearity can significantly improve the speed and accuracy of RUL prediction. Consequently, the subsequent section presents an introduction to the key signal pre-processing methods.

2.10.1. Empirical Mode Decomposition (EMD)

EMD, introduced by N. E. Huang et al. in 1998, is a time-frequency signal processing technique that is adaptive and automatic. It decomposes non-stationary signal data into a finite number of high-frequency intrinsic mode functions (IMFs) and a low-frequency residual sequence (RES) (Zhou et al., 2016). In RUL prediction models, the IMF components are commonly used to capture capacity regeneration and local fluctuations in battery

capacity degradation, whereas the residual component represents the overall capacity degradation trend.

EMD has two main applications in RUL prediction. The first category involves disregarding the IMFs to address the issue of capacity regeneration. In this approach, the residue obtained from EMD exhibits a strong correlation with the original data, allowing it to effectively reflect changes in battery capacity while filtering out noise. The second category considers both capacity regeneration and random fluctuations, employing hybrid ML methods (Liu et al, 2020). The IMFs are fitted using the first ML method (ML1) to capture local regression and fluctuations, while the residual sequence is analyzed using the second ML method (ML2). The final RUL prediction combines the results from both ML methods. While this approach may enhance accuracy, it requires additional computational resources due to the inclusion of extra IMFs.

The signals obtained through EMD decomposition consist of two main components: extracted features used for RUL prediction.

EMD is efficient and doesn't require predefined basis functions but suffers from mode mixing. To overcome this, EEMD was introduced as a noise-assisted method. EEMD averages IMF components from multiple trials with added white noise, reducing mode aliasing effects.

2.10.2. Variational Modal Decomposition (VMD)

VMD algorithm decomposes signals adaptively into sub-signal IMFs, overcoming EMD limitations. It improves RUL prediction accuracy by decomposing battery degradation data into trend and fluctuation series. For NASA 5# battery, VMD enhanced RUL prediction accuracy from 78% to 93% before and after denoising (Fan et al., 2019).

2.10.3. Box-Cox Transformation

Due to its capability to enhance the linearity between features and RUL by improving the additivity, normality, and homoscedasticity of observations, BCT is commonly employed. The basic form of BCT is:

$$Y_{BC}(Y,\lambda) = X\beta + \sigma e$$

where,

$$Y_{BC}(Y,\lambda) = \begin{cases} \frac{Y^{\lambda}-1}{\lambda} if \lambda \neq 0 \\ \log(Y) \text{ if } \lambda = 0 \end{cases}$$

Box-Cox Transformation (BCT) used in RUL prediction to transform the observation y. The parameter k is determined using profile likelihood and a goodness-of-fit test. The transformed variables (HI) are subjected to Pearson correlation analysis to evaluate their linear relationship with independent variables (extracted features) (Fan et al., 2019). BCT is also employed to enhance the linear relationship between features and capacity, as depicted in Fig. 6. The transformed variables are then utilized in an ML method. A summary of BCT applications, including the independent variables, transformed variables, and prediction variables.

2.10.4. Wavelet Decomposition Technology (WDT)

Wavelet analysis (WDT) decomposes non-stationary signals into an approximate part and a detailed part using scaling and wavelet functions. The approximated part captures the global degradation trend, while the detailed part reveals local regeneration patterns and fluctuations. In RUL prediction, WDT separates global degradation and local regeneration in battery capacity series. The approximated part is used in the first ML method, the detailed part in the second, and the results are combined iteratively for RUL determination (Wan et al., 2017)

In practical applications, the discrete wavelet transform (DWT) is used to decompose the signal into different scales, capturing global degradation and local regeneration in the SOH series. Wang et al. developed an RUL prediction function based on DWT-decomposed terminal voltage. The choice of wavelet function and decomposition levels affect the model's performance. Wavelet packet decomposition (WPD) enhances wavelet transform by

eliminating noise in charge-discharge cycle data. WPD decomposes both low-frequency and high-frequency sub-bands. Chen et al. introduced wavelet packet energy entropy (WPEE), combining WPD with information entropy theory.

2.11. Machine Learning Methods

RNN emerged as the earliest and most widely adopted method, followed by ANN. SVR and AR were also used initially. Further analysis will focus on highly popular algorithms with over 10 occurrences.

Non-probabilistic Method

2.11.1. Recurrent neural network

RNN algorithm is suitable for processing time series data because its hidden layer makes the current output related to the previous output (Kwon et al., 2020).

The battery degradation data collected from charging and discharging cycles can be seen as time series data, making RNN suitable for RUL prediction.

2.11.2. Echo state network:

The ESN is composed of an input layer, a reservoir layer, and an output layer. Win is the input weight matrix. Wre is the internal connection weight matrix? Wback is the feedback matrix returning the output of ESN to the reservoir layer. Wout is the output weight matrix. The reservoir layer consists of many sparsely connected neurons and thus has the function of short-term memory by adjusting these neurons (Ji et al., 2021).

2.11.3. Long short-term memory:

LSTM uses gate structure, a method that can control the information flow, to solve the vanishing gradient problem. In the input features of LSTM, the one-to-one structure is replaced by a many-to-one structure. The many-to-one structure refers to the use of multiple input vectors simultaneously, such as current, voltage, and temperature, while the one-to-one structure uses a single input vector of historic capacity (Zhang et al., 2017). It has been

proven that many-to-one structures can improve RUL prediction accuracy even in the presence of capacity regeneration.

(Zhang et al., 2018) proposed a BLS-LSTM model which combined LSTM and Broad Learning System (BLS) which has the additional enhancement nodes from the initial feature input. Based on this, the more effective feature can be extracted. Furthermore, with the adoption of the adaptive sliding window, LSTM can learn the local fluctuations and the long-term dependencies simultaneously.

Additionally, the Bidirectional LSTM (Bi-LSTM) is another type of LSTM that has been used in RUL prediction. Bi-LSTM learns both forward and backward using two independent LSTM layers. Yang et al. employed Bi-LSTM information to extract more information from the same signal because they believed that the battery deterioration series signals were related to the current time in both forward and backward directions. In order to predict RUL, Yuhuang et al. used Bi- LSTM in the discriminator of the generative adversarial network (GAN).

Gate optimisation, algorithm combinations (such as RUL prediction based on signal decomposition, measurement variable as filter algorithm, and combination with CNN), ensemble learning, and transfer learning are at the heart of LSTM research.

Using LSTM gate optimisation to increase RUL accuracy is a successful tactic. An essential component of the LSTM is the information that the gates regulate. In order to simultaneously pick old and fresh information, Li et al. connected the input and forget gate by a fixed connection. In order to gather additional data and lower the error, they also conducted the element-wise product between the new inputs and historical cell state and added a peep-hole connection to the output gate. In order to properly regulate the output information of the memory cell (Cheng et al., 2021).

The main components of an LSTM algorithm combination are the prediction of a decomposed signal using various signal decomposition methods, such as EMD, as well as the measurement variable of PF or KF, and combining with CNN using various connection methods. More information has been covered in the 'Signal pre-processing' section above. When handling non-linear problems, UKF or KF exhibit advantages. The issue is that using

the battery electrochemical model makes it difficult and inaccurate to obtain the measurement values. Cui et al. implemented LSTM and RBF NN in UKF to achieve this, where RBF NN is utilized to approximate the classical exponential model as a state transition and LSTM estimated capacity is employed as the measurement vector. By implementing LSTM to train the deterioration model and propagating each particle using the state transition that was learned by LSTM, Zhang et al. employed PF to update the exponential model of LIBs (Yang et al., 2020).

The future measurement value of PF was similarly determined by Hu et al. using the LSTM prediction mean values. The LSTM model was iterated 100 times, and the mean was calculated using the Monte Carlo dropout approach. In order to extract spatial and temporal information from the data, LSTM and CNN are frequently coupled. The 'Convolutional Neural Network' section of subsection 2.3.2.4 goes into more detail. Series and parallel connections are included in the combined way. Using ensemble learning, a more accurate forecast can be made. Liu et al. proposed to ensemble LSTM sub-models with Bayesian model averaging (BMA) to gain greater accuracy and uncertainty representation (Liu et al., 2019).

The LSTM sub-models used several sub-datasets that were created as a result of the degrading of training data. In order to forecast the RUL, Song et al.'s team and Wang's team both presented an ensemble model based on stacked LSTM, where prediction values from each LSTM were used as input to the ensemble model. The measurement for the PF updation likewise came from the TL-LSTM value (Pan et al., 2020).

2.11.4. Gated recurrent unit:

The GRU is similar to the LSTM, but it features two gates that combine the forget and input gates of the LSTM to create an update gate that is used to manage how much the prior information influences the current information and uses a reset gate to manage how much the previous information is ignored. GRU is quicker to train while still being able to reduce both exploding and disappearing gradients because it has a simpler structure than LSTM (Song et al., 2018).

The study of GRU in batteries Deep information extraction, transfer learning, and fundamental application are the three main areas of RUL's concentration (Wang et al., 2021).

With four hidden layers and a maximum inaccuracy of 11 cycles for the RUL prediction on the NASA B0006 battery, Song et al. established the battery RUL prediction method using the GRU. In order to optimize the network, Ardeshiri, Ma, and colleagues used the ADAM algorithm, which can cut down on training time and the influence on learning rate. To address overfitting, they also suggested an early stopping approach. Wei et al. employed the Monte Carlo Dropout (MC_dropout) method with GRU to predict RUL, with MC primarily used to produce the probability distribution and 95% confidence interval of the RUL prediction point. To prevent over-fitting, the drop-out approach is used with the weights of the concealed layers (Che et al., 2021).

(Wang et al., 2021) suggested an adaptive sliding window GRU to get additional knowledge about battery deterioration. The memory units and gate mechanism that define the GRU let it to capture the long-term dependencies of its feature. As the sliding window's size changes in response to the feature's variation, the adaptive sliding window can pick up on local regenerations and fluctuations. IRes2Net-BiGRU-FC was utilized by Tang et al. to forecast various frequency data using CEEMDAN. Bidirectional Gated Recurrent Unit (BiGRU) in IRes2Net-BiGRU-FC may concurrently record battery capacity information in the past and the future, increasing the variety of characteristic information.

For online applications, transfer learning can cut down on training time and training data. Based on having comparable degradation tendencies, a combination of transfer learning and GRU was suggested to forecast the online RUL of batteries. By using Euclidean distance, the similarity of deterioration trends for several batteries is compared.

2.11.5. Support vector regression

Because it can map the non-linear connection between input and output data based on structural risk reduction, SVR is appropriate for RUL prediction (Yu et al., 2020). The secret is turning a challenging nonlinear issue into a straightforward linear one utilising the kernel trick. The SVR model's generic form is stated as follows:

$$f(\boldsymbol{x}) = \langle \omega, \boldsymbol{x} \rangle + b$$

where the input value is x. The target value is f (x). The vector of weight is w. The bias term is represented by b. Zhao et al. used the SVR in conjunction with feature vector selection (FVS) to predict RUL. By eliminating redundant data, the FVS-SVR technique can reduce the search space for support vectors. In their study, two characteristics were extracted: the time interval of an equal charging voltage difference (TIECVD) and the time interval of an equal discharging voltage difference (TIEDVD) (Zhou et al., 2018).

The band size, which is governed by epsilon, the penalty factor, and any kernel function parameters are the most typical hyper-parameters. Differential evolution, particle swarm optimization, whale optimization (WOA), genetic algorithms, grey wolf optimization (GWO), improved ant lion optimization (ALO), artificial bee colony (ABC), improved bird swarm (IBS), grid search have all been used to optimize the hyper-parameters.

2.11.6. Auto-Regression

A time series model called AR is used to forecast the series' values in the near future. For time series analysis, it offers the benefit of long-term trend prediction (Liu et al., 2015). For instance, Vilsen et colleagues employed AR to record the internal resistance of the battery's long-term behavior change. Its primary purposes are as follows:

$$x_t = \phi_1 x_{t-1} + \phi_2 x_{t-2} + \cdots + \phi_p x_{t-p} + u_t$$

Where p signifies the white noise sequence; x_t represents the battery capacity value at a given time; represents the number of autoregressive terms (Vilsen et al., 2020). To improve prediction accuracy, the order of the AR model must be determined. Popular ways for doing this include the Box-Jenkins method, the Akaike information criterion (AIC), and the Bayesian information criterion (BIC).

Because the deterioration rate often accelerates as the cycle number rises, the degrading process for lithium-ion batteries is nonlinear. The nonlinear accelerated degradation factor, which is connected to the degradation cycle and time-varying, is added to improve the nonlinear capability of the AR model. The function of the prediction step is how the factor is defined in practice. It is referred to as the nonlinear deterioration AR (ND-AR) model as

a result. After reviewing the ND-AR model, Guo et al. modified the factor and proposed the NSDP-AR model, in which the factor is connected to the current percentage of life-cycle length, and the related parameters in the formula to calculate the factor are obtained using an EKF (Liu et al., 2014).

To enhance the long-term prediction and iterative prediction of PF, AR models are typically coupled with various PF, where the output of the AR model is utilized as the observed or measured value in PF. The iterative nonlinear deterioration autoregressive (IND-AR) model was proposed in addition to the ND-AR model for this. The regularised PF (RPF) and the PF based on the deformed double exponential empirical deterioration model are additional options to the standard PF. Additionally, RVM and AR models have been merged, where the uncertainty quantification is based on the calculation of the mixture distribution.

2.11.7. CNN

According to Fig. 8, CNN typically consists of three layers: a convolutional layer, a pooling layer, and a fully connected layer. A filter is used to execute a convolution process to extract spatial information from the preceding layer (Zhou et al., 2020). The extracted neurons that conduct the convolution are commonly referred to as the receptive field and the convolution:

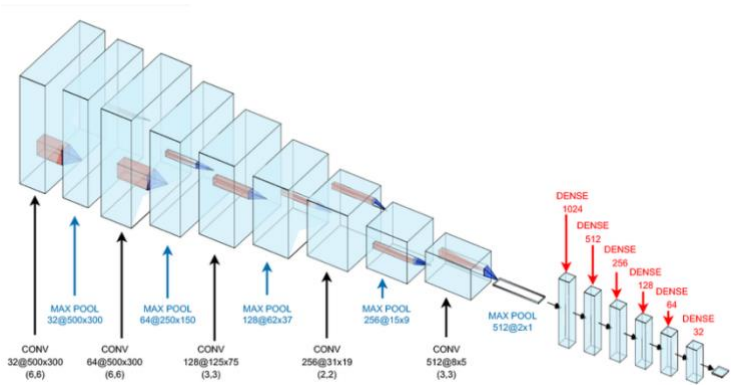


Figure 8 The general architecture of CNN (Ciccarelli et al., 2023)

The fully connected layer can then be utilized for regression once the pooling layers have been used to extract local features. The basic goal of utilizing CNN is to extract the spatial information or relationships from the inputs and to condense them into a smaller spatial dimension. Hsu et al. merged these new features with features that were created by humans

after using two CNNs to lower the dimension of the input data. With only one cycle used for testing, their deep neural network's (DNN) MAPE to predict battery RUL is 6.46%. One semi-supervised technique for estimating capacity deterioration was put out by Rui et al. Without corresponding capacity labels, capacity was estimated using the CNN to extract features from the raw EIS data (Xiong et al., 2023).

Transfer learning can compensate for a small dataset and increase the effectiveness of CNN training. In the study by Shen et al., eight deep convolutional neural networks (DCNN) models were pre-trained using 10-year daily cycle data of eight implantable Li-ion batteries as the source dataset. The target domain was then utilized to create eight DCNN with transfer learning (DCNN-TL) models using the obtained parameters of the pre-trained DCNN models. The DCNN with Ensemble Learning and Transfer Learning (DCNN-ETL) is an ensemble model that incorporates these DCNN-TL models. Compared to GPR, RF, and DCNN, the suggested DCNN-ETL approach is more accurate and robust.

Future research should continue to focus on how to estimate RUL using little datasets quickly and accurately.

2.11.8. Extreme learning machine

A feedforward neural network called an extreme learning machine (ELM) has a single hidden layer in between the input layer and the output layer. Given that the weights and biases between the input and hidden layers are produced at random, no training is required. In other words, the only parameters that require training during the learning process are the output weights (Razavi-Far et al., 2019). The generalized Moore- Penrose matrix inverse is typically used to train the output weights, which has the advantages of fewer training parameters and quick learning. ELM may be used for quick and online RUL prediction. ELM has a wide range of uses in the field of predicting battery RUL. An integrated ELM-based framework for RUL prediction in the absence of data or features was introduced by Roozbeh et al. According to Chen et al., who used the two-phase Wiener process to derive the mathematical formula for RUL and an ELM to construct the feature of RUL and detect the change point of the model by the feature adaptively, the degradation path of the lithium battery can be roughly described as having two different degradation rates (phases).

The initial optimization of the input parameter, online sequential ELM (OS-ELM), and quasi-RNN are the primary areas of ELM research (Fan et al., 2019).

The input weights and the bias of the ELM are provided, as was previously described. In order to increase accuracy, initial weights and bias are optimized using PF, improved PSO with mutation factors, Genetic algorithm ant algorithm (GAAA), and hybrid grey wolf optimization (HGWO), which combines differential evolution (DE) and grey wolf optimization (GWO) (Fan et al., 2019).

The prediction time of RUL will be shortened if the ELM model parameters can be updated using the new data as opposed to the retraining model. An online sequential-extreme learning machine (OS- ELM) is suggested in order to decrease the amount of training data needed to train ELM. The initialization phase and the sequential learning phase make up this process. The goal of the sequential learning phase is to update the output weights established in the first phase using fresh data samples.

The whole link between the input layer and hidden layer is replaced by convolution and pooling based on the OS-ELM in (Tang et al., 2019) faster online sequential pooling extreme learning machine (OS-PELM). Forgetting Online Sequential Extreme Learning Machine (FOS-ELM) was another idea put out by Fan et al. It has a forgetting mechanism that gradually ignores any old data in the learning process and the dynamic incremental update function used when new data is received.

(Zhang et al., 2022) suggested a deep adaptive continuous time-varying cascade network based on extreme learning machines (CTC-ELM) in the case of little data. Because each sub-networks input is made up of the output of the previous network and the original time series data, a CTC-ELM is like an RNN in terms of its time-varying nature. However, it still has ELM's speed edge.

Despite the ELM's many benefits, its shallow structure might make it difficult for it to fully capture the characteristics of high-dimensional data and may prevent it from meeting the demands of battery datasets that are growing in size. Ma, therefore, suggests the broad learning extreme learning machine (BL-ELM).

Probabilistic Methods

2.11.9. Relevance vector machine

A sparse probability model built on Bayesian inference is known as RVM. Its model can be stated as Equation (4) and shares a general shape with SVR. Unlike SVM, RVM is unconstrained by Mercer's theorem. RVM may create many kernel functions and has the benefit of uncertainty expression. Additionally, RVM requires less processing because there are fewer hyper-parameters and fewer relevance vectors (Feng et al., 2021).

A common choice for features in the RVM used for RUL prediction is the sample entropies of current and voltage, charging current change rate in the CV stage (CCCR_CV), constant current charge time (CCCT) and ambient temperature, the duration of equal discharging voltage difference (DEDVD) and the duration of equal charging voltage difference (DECVD). The Kalman filter (KF) or unscented Kalman filter (UKF) is typically incorporated with RVM. RVM was utilized by Zheng et al. to forecast the future development of UKF residuals (Zhang et al., 2019).

In order to correct the UKF prediction result, RVM was also utilized to predict the new error series. Whenever possible, RVM is used to combine the outputs of various methods without probability distributions or from various prediction starting points in order to improve the performance of the overall forecast.

According to statistics in this review, the evolution of the research core of RVM has three main components, which are kernel parameter optimization, kernel function combination, and long prediction performance enhancement (Chen et al., 2021).

Typically, an RVM's kernel parameters are chosen using empirical data. The Gaussian kernel function with 5 kernel parameters is the kernel function of the optimized RVM. Cai et al. developed the artificial fish swarm method (AFSA) to locate the ideal kernel parameter in order to increase prediction accuracy (Cai et al., 2017). RVM's kernel function bandwidth was optimized by Zhou et al. using the PSO approach, and their model's average error was less than 2.18%.

Another method for enhancing RVM's kernel function is multiple kernel learning, which combines the benefits of several different kernel functions. The selection of the kernel function's weights is crucial to this strategy. While Sun et al. employed the grid search method to optimize the parameters of their multiple kernel RVM, Zhang et al. used the PSO approach to achieve multiple kernel RVM. Some studies attempted to concurrently achieve kernel parameter optimization and kernel function combination. In, a discrete PSO method was used to optimize the ensemble weight coefficient of the various kernel functions and parameters, realizing the selection of some efficient kernels and the optimization of kernel parameters for RVM (Yang et al., 2016).

This discrete PSO method's prediction accuracy was superior to that of the single kernel RVM and the RVM model that included all kernel functions. The Cuckoo Search (CS) algorithm was used by Chen et al. to identify each kernel function's ensemble weights as well as all of its kernel parameters. It was discovered that CS outperformed the discrete PSO algorithm in terms of effectiveness and accuracy (Feng et al., 2019).

Poor long-term prediction performance is a significant limitation of RVM. The fundamental cause is that in a single-step prediction, the relevance vectors and coefficients matrix of the RVM cannot update simultaneously. An incremental learning technique is introduced to increase the accuracy of RVM's multi-step prediction. The key to this strategy is that the input data for RVM includes both the relevance vector and a new online data sample.

By using each step's results to update the training dataset and retraining the RVM model based on the new and online training dataset, a KF can be utilized to optimize the RVM result for greater long-term prediction accuracy. To increase the stability of lengthier predictions, (Zhao et al., 2017) employed Deep Belief Network (DBN) to extract the features as the inputs of RVM. For long-term prediction taking capacity regeneration into consideration, the RVM and grey model (GM) are alternately utilized in.

In the future, generating RVs with RVM and then feeding them into other algorithms with higher long-term prediction performance will be the optimal method. Additionally, the RVM hardware implementation is crucial, just like the suggested architectural design in.

2.11.10. Gaussian process regression

The GPR model is a probabilistic, nonparametric approach. It can be thought of as a grouping of a small number of random variables with a shared Gaussian distribution whose mean and covariance are described (Dong et al., 2021).

In order to model RUL with GPR, relevant features are typically extracted as input and fed into the GPR to produce the RUL. Peaks, regional areas, slopes, and other geometric features from incremental capacity analysis (ICA) curves, as detailed in, are the most typical features (Zhang et al., 2020).

Kernel function improvement, model combination, and pseudo-transfer learning are the primary research areas for GPR.

As illustrated by (Liu et al., 2019) who combined the Arrhenius law and a polynomial equation with the covariance function in the kernel function, adding electrochemical or empirical aspects can be highly beneficial.

With an MAE and RMSE of less than 0.07 Ah (0.3%) and 0.09 Ah (0.4%), respectively, the enhanced kernel function may reflect temperature, DOD, and capacity. In order to eliminate the irrelevant inputs to the GPR, they further updated the fundamental squared exponential (SE) function acting as a covariance function with the automatic relevance determination (ARD) (Richardson et al., 2017).

Richardson et al. employed an exponential model to improve the predictive performance of basic Gaussian Process Regression (GPR) by considering battery degradation as a mean function rather than zero. Another approach to enhance kernel functions is to combine different types of kernels. Liu et al. introduced the square exponential covariance function and periodic covariance function to better describe capacity degradation and regeneration phenomena, respectively. Similarly, Peng et al., 2018) proposed a hybrid GPR that combines these kernels to enhance the accuracy of Remaining Useful Life (RUL) prediction. In a study evaluated 10 different compound kernel functions to identify the one with the best performance. To address the challenge of selecting the kernel function, Neu- ral Gaussian Process (NGP) was proposed, which leverages the advantages of neural network function approximation.

2.12. Comparison of RUL Prediction Methods

The accuracy of the ML algorithms, as well as algorithmic traits such as online updating performance (a time-varying characteristic), uncertainty quantification, generalization, and ease of implementation, are assessed.

However, practically every system for predicting battery RUL has limits. Long-term dependency prediction is not a good fit for the SVM method, and its accuracy has never been strong. It occasionally exhibits overfitting or a non-convergence trend, especially when full-life data are lacking (Rouhi Ardeshiri, & Ma, 2021). Additionally, SVM is unable to capture the capacity regeneration phenomena of batteries, and when employed with various batteries, the prediction error fluctuation is significant (Mansouri et al., 2017). The capacity renewal phenomena of batteries cannot be captured by the GPR approach either, and its prediction curves occasionally exhibit little resemblance to actual data.

2.13. Insights and Future Directions

Over the past decade, significant advancements have been made in utilizing machine learning (ML) algorithms for predicting the RUL of lithium-ion batteries. This review provides a systematic summary of the key research areas and development trends in common ML methods for battery RUL prediction. It also briefly discusses approaches to extend battery lifespan and analyzes the potential of using RUL prediction to achieve this goal.

The study analyzes the ten most frequently used ML algorithms for RUL prediction between 2012 and 2022, along with their occurrence frequency in published papers. Among these algorithms, Recurrent Neural Network (RNN) is the most commonly employed, appearing in 61 papers from the indexed database. The four most frequently used signal processing methods and their application in conjunction with ML methods are detailed. Unified figures illustrating the evolving research focus of the top-used ML algorithms over the years are provided to facilitate a quick and clear understanding of their development.

Overall, research on these algorithms primarily concentrates on parameter optimization, algorithm fusion, structure design, and transfer learning. Building upon the analysis of these algorithms, specific improvement directions and possibilities for RUL prediction are

outlined, including early prediction, modeling local regeneration, integrating physical information, employing generalized transfer learning, and exploring hardware implementation. By optimizing the charging profile based on online RUL prediction results, end-users can extend the battery's lifetime. The authors aim to assist researchers in comprehending the core developments of different ML algorithms in RUL prediction and inspire the creation of more valuable algorithms in the future. Additionally, the review's section on extending battery lifespan through RUL results intends to provide insights and inspiration for practical applications.

3. INTRODUCTION TO DATA

3.1. Real World Dataset

Present-day applications for lithium-ion batteries are manifold. Significant markets include cell phones, portable computing devices (PCs), electric vehicles, and grid storage (Pramanik et al., 2019). To meet system requirements in high-output voltage applications, it is common to connect multiple lithium-ion cells in series. For vehicle applications, this number can reach up to one hundred, while two to four cell chains are typically sufficient for notebook PCs. Grid storage may require even more extensive series of cells. However, this practical scalability in battery pack design comes with certain disadvantages.

Current lithium-ion batteries are utilized in a variety of contexts. Key consumers of lithium-ion batteries consist of electric vehicles, cell phones, portable personal computers (PCs), and

grid storage (Hesse et al., 2017). Generally, in order to satisfy system requirements for high output voltage applications, a series assembly of lithium-ion cells is utilized.

Typically, two to four cell strings are necessary for notebook PCs and other devices, whereas this number can reach one hundred for vehicle applications. Grid storage may require series of components that are even more extensive. The favorable aspect of scalability in battery pack design is not without its limitations. One such fact is that the durability of a series of lithium-ion cells is constrained to the strength of its least robust component.

The existence of variations among cells during the aging process has been demonstrated thus far. Predicting the cycle-life of millions of cells in a timely and resource-efficient design is an exceedingly challenging task, if not impossible one.
This is problematic because an unquantified or excessively large intrinsic degradation variability has numerous repercussions. It complicates the task of comparing competing cell designs and determining which one is preferable for battery designers. Battery integrators are confronted with significant obstacles concerning warranty terms and charge-balancing strategies (Velho et al., 2017).
As of now (Rohr et al., 2017) it has been established that variations occur among cells as a result of aged cells. Determining the cycle-life of millions of cells in a timely manner while conserving resources is an incredibly difficult, if not impossible, undertaking.
This presents a challenge as an unmeasured or excessive degree of intrinsic degradation variability can result in an extensive list of consequences. For battery designers, this complicates the process of comparing contending cell designs and determining which one is superior. Battery integrators encounter substantial challenges with respect to charge-balancing strategies and warranty conditions (Velho et a., 2017).
The main study conduced by (Devie et al., 2018) shows 51-cell of industrial lithium-ion cells. The research revealed that before cycling, the properties of the cells exhibited a narrow dispersion: the average C/5 capacity was 2,863 Ah, coupled with a standard deviation of 0.4%; the C/2 capacity was 99.3% of the value observed at C/5, with a standard deviation of 0.3%, suggesting an exceptional rate capability; and the average resistance was 73.2 mΩ, with a standard deviation of 3.6%. During cycling, this collection of metrics was monitored

and it was discovered that the resistance increased by 80% to 112.3 mΩ with a standard deviation of 5.0% that remained relatively constant.

To improve battery management and RUL forecasting, it is critical to select the most relevant data sets with care. Since Li-ion battery research is a popular area of study, many organizations have made datasets available for various battery models. Utilizing these datasets can improve the estimation of battery states and the prediction of RUL, while also reducing the time needed to generate new datasets (Deng, 2015). In this thesis, we acquired real-world dataset from GitHub. The dataset tracks numerous performance metrics of rechargeable batteries over their charge-discharge cycles, presenting significant insights into their operational efficiencies and RUL.

A concise overview of the various techniques employed in EV charging is initially presented to facilitate an understanding of the process's complexities. Analyzing battery charging data is complicated due to the intricate charging patterns of most modern advanced fast-charge batteries. A standard complete charging procedure comprises three segments: pre-charging, constant-current charging, and constant-voltage charging. Pre-charging is performed to prepare the battery for high-current rapid charging.

3.2. Synthetic Generation

This chapter details the process of generating synthetic data using the Gaussian Copula method from the ‘sdv’ library. It includes steps for data preprocessing, model fitting, synthetic data generation, and subsequent analysis and visualization.

3.2.1. Importance of Synthetic Data Generation

Generating synthetic data has become crucial in various fields, particularly when dealing with sensitive or limited datasets. The Synthetic Data Vault (SDV) library, developed by the MIT Data To AI Lab, is a powerful tool designed to create high-quality synthetic data that retains the statistical properties of the original data (Zhang et al., 2022). This approach addresses several key issues:

1. **Privacy Preservation:** Real datasets often contain sensitive information that cannot be shared freely due to privacy concerns. Synthetic data mimics the original data without exposing real individual records, thus safeguarding privacy (Endres et al., 2022).

   The framework known as Synthetic Data Vault with Gaussian Copula (SDV-G) produces synthetic data through the application of a multivariate model that is constructed by intersecting multiple tables within a relational database. Input for the modeling process of the entire database comprises the data tables and the metadata that corresponds to each table.

   SDV's framework presents the benefit of simple implementation across all dataset formats. The format of both input and generated synthetic data is dataframes. Additionally, it should be mentioned that Label Encoding is applicable to every SDV model. SDV-G is directly implemented by designating the generator with the Gaussian Copula model (Endres et al., 2022).

2. **Data Augmentation:** In scenarios where the available data is limited, synthetic data can augment the dataset, providing additional samples for training machine learning models. This is particularly useful in fields like healthcare, finance, and autonomous driving, where gathering real data can be expensive or impractical.

3. **Testing and Development:** Synthetic data provides a safe environment for testing and developing new algorithms. It allows developers to experiment without the risk of compromising sensitive information or violating data usage agreements.
   The statistical significance of such analyses must be adjusted accordingly, given that conclusions drawn from data analysis and hypothesis testing are inevitably less accurate when synthetic data is utilized as opposed to real data.

   Critically, the objective is for the process of developing a model using synthetic data to mirror the process of developing a model using real data. For instance, when comparing two hyperparameter options, the comparison should precisely replicate the one conducted using the real data (Jordon et al., 2022). Subsequently, the efficacy

of the synthetic dataset is assessed based on the correspondence between the performance rankings of models implemented on the synthetic data and those predicted using the actual data.

4. **Compliance with Regulations:** Regulations such as GDPR and HIPAA impose strict guidelines on data usage and sharing. Synthetic data can help organizations comply with these regulations by enabling data analysis and sharing without breaching legal constraints.

   The research demonstrated that implementing data Augmentation (DA), a regularization scheme that augments the dataset artificially by incorporating more invariant examples through label-preserving transformations (Taylor et al., 2018).

5. **Improved Model Performance:** By augmenting pre-existing datasets with synthetic data, one can facilitate interpolation between cycling conditions not captured in the experimental data, thereby improving the performance of trained models. This interpolation phase is crucial in data-driven approaches that enable predictions "beyond the distribution" of the empirical data.

   Synthetic data exhibits a variety of advantages that natural data lacks, generating it an appealing alternative across numerous domains.

   In comparison with organic data, synthetic datasets are comparatively simple to obtain and have the capability to provide information in unusual or challenging factors, thus mitigating diversity concerns in specific datasets. Moreover, this technology can efficiently overcome concerns regarding privacy,

   In our research, the Synthetic Data Vault (SDV) is a Python library that simplifies the process of generating synthetic tabular data. A variety of machine learning

algorithms are utilized by the SDV to identify patterns in your actual data and simulate them in synthetic data. The SDV library facilitates the assessment of synthetic data through a comparative analysis with real-world data. Begin with generating a thorough report (Hao et al., 2024).

However, synthetic data needs to satisfy two criteria:

- It must statistically closely resemble the original data to maintain data scientists' interest and ensure realism.
- It must resemble the original data formally and structurally to facilitate the reusability of software built upon it.

3.2.2. Preprocessing and Assigning Unique Device IDs

Before generating synthetic data, it is crucial to preprocess the dataset by assigning unique 'DeviceID's to different devices or cycles based on the Remaining Useful Life (RUL) values.

SDV

```
[3] from sdv.single_table import GaussianCopulaSynthesizer
    from sdv.metadata import SingleTableMetadata
    # Initialize the 'DeviceID' column with zeros
    data['DeviceID'] = 1

    # Variable to hold the current device ID
    current_device_id = 1

    # Iterate through the rows to assign device IDs based on the logic described
    for i in range(1, len(data)):
        # If the RUL at the current row is higher than the RUL at the previous row,
        # it indicates the start of a new device/cycle
        if data.iloc[i]['RUL'] > data.iloc[i - 1]['RUL']:
            current_device_id += 1
        data.at[data.index[i], 'DeviceID'] = current_device_id
    data['DeviceID'][0]= 1
    # Now, let's check the first few rows to verify the 'DeviceID' assignment
    data.head()
```

	Unnamed: 0	Cycle_Index	Discharge Time (s)	Decrement 3.6-3.4V (s)	Max. Voltage Dischar. (V)	Min. Voltage Charg. (V)	Time at 4.15V (s)	Time constant current (s)	Charging time (s)	Total time (s)	RUL	DeviceID
0	0	1.0	2595.30	1151.488500	3.670	3.211	5460.001	6755.01	10777.82	14648.45	1112	1
1	1	2.0	7408.64	1172.512500	4.246	3.220	5508.992	6762.02	10500.35	19124.28	1111	1
2	2	3.0	7393.76	1112.992000	4.249	3.224	5508.993	6762.02	10420.38	19029.51	1110	1
3	3	4.0	7385.50	1080.320667	4.250	3.225	5502.016	6762.02	10322.81	18923.65	1109	1
4	4	6.0	65022.75	29813.487000	4.290	3.398	5480.992	53213.54	56899.65	104171.80	1107	1

Figure 9 Preprocessing Results

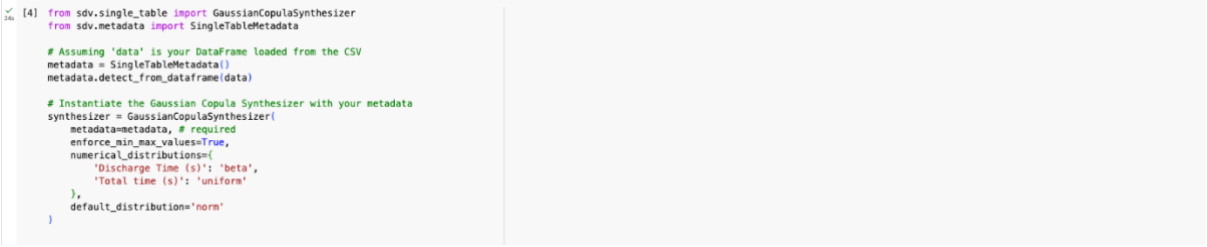
Explanation:

- The 'DeviceID' column is initialized to ones.
- A 'current_device_id' variable tracks the current device or cycle.

- As the loop iterates through the dataset, if the current row's RUL is higher than the previous row's RUL, it indicates the start of a new device or cycle, and the current_device_id is incremented.
- Each row is then assigned the appropriate 'DeviceID'.
- This preprocessing ensures that each cycle or device is uniquely identified, maintaining the contextual integrity of the dataset.

Defining Metadata and Initializing the Synthesizer

Next, we define the metadata and initialize the Gaussian Copula Synthesizer for synthetic data generation.



```
[4] from sdv.single_table import GaussianCopulaSynthesizer
    from sdv.metadata import SingleTableMetadata

    # Assuming 'data' is your DataFrame loaded from the CSV
    metadata = SingleTableMetadata()
    metadata.detect_from_dataframe(data)

    # Instantiate the Gaussian Copula Synthesizer with your metadata
    synthesizer = GaussianCopulaSynthesizer(
        metadata=metadata, # required
        enforce_min_max_values=True,
        numerical_distributions={
            'Discharge Time (s)': 'beta',
            'Total time (s)': 'uniform'
        },
        default_distribution='norm'
    )
```

Figure 10 Initialization of Synthesizer

Explanation:

- Metadata for the table is detected from the dataframe.
- The 'GaussianCopulaSynthesizer' is initialized with the metadata. Additional parameters are set:
- 'enforce_min_max_values=True' ensures generated values are within the observed range.
- Specific numerical distributions are assigned to certain columns, with a default normal distribution for others.

Fitting the Synthesizer and Generating Synthetic Data

The synthesizer is then fitted to the dataset, and synthetic data is generated.

```
# Fit the synthesizer to your dataset
synthesizer.fit(data)

# Generate synthetic data, adjust num_rows as needed
synthetic_data = synthesizer.sample(num_rows=1000000)  # Generates 100 rows of synthetic data
sorted_synthetic_data = synthetic_data.sort_values(by=['DeviceID', 'RUL'], ascending=[True, False])
```

```
/usr/local/lib/python3.10/dist-packages/sdv/single_table/base.py:86: UserWarning: We strongly recommend saving the metadata using 'save_to_json' for replicability in future SDV versions.
  warnings.warn(
```

Figure 11 Fitting and Generating the Synthetic Data

Explanation:

- The ‘fit’ method trains the synthesizer on the original dataset.
- The ‘sample’ method generates synthetic data, here creating 100,000 rows.
- The synthetic data is sorted by ‘DeviceID’ and ‘RUL’ to maintain logical ordering.

Filtering and Sorting Synthetic Data

Post-generation, the synthetic data is filtered and sorted to ensure consistency and validity.

```
[5] # Filter rows where Decrement is positive
    positive_decrement_df = sorted_synthetic_data[sorted_synthetic_data['Decrement 3.6-3.4V (s)'] > 0]

    # Sort by DeviceID, Cycle_Index, and RUL in descending order
    sorted_df = positive_decrement_df.sort_values(by=['DeviceID', 'Cycle_Index', 'RUL'], ascending=[True, True, False])

    # Group by DeviceID and Cycle_Index and then randomly select one row per group
    selected_df = sorted_df.groupby(['DeviceID', 'Cycle_Index']).apply(lambda x: x.sample(1)).reset_index(drop=True)
```

Figure 12 Filtered Synthetic Data

Explanation:

- Rows with positive ‘Decrement 3.6-3.4V (s)’ values are filtered.
- The data is sorted by ‘DeviceID’, ‘Cycle_Index’, and ‘RUL’.
- One row per group of ‘DeviceID’ and ‘Cycle_Index’ is randomly selected to reduce redundancy and ensure variety.

Ensuring Decreasing RUL Values

A custom function ensures that RUL values decrease for each DeviceID.

```
# Ensure that RUL values are decreasing for each DeviceID
def filter_decreasing_rul(group):
    group = group.sort_values(by='Cycle_Index')
    valid_rows = []
    last_rul = float('inf')
    for index, row in group.iterrows():
        if row['RUL'] < last_rul:
            valid_rows.append(row)
            last_rul = row['RUL']
    return pd.DataFrame(valid_rows)

final_df = selected_df.groupby('DeviceID').apply(filter_decreasing_rul).reset_index(drop=True)
```

Figure 13 Reducing RUL Values

Explanation:

- The function ‘filter_decreasing_rul’ ensures that for each ‘DeviceID’, the RUL values decrease as expected.
- The function iterates over each group, retaining rows where the RUL decreases.

Saving and Loading Synthetic Data

The cleaned synthetic data is saved to a CSV file and reloaded for analysis.

```
[6] final_df.drop(['Unnamed: 0','DeviceID'],axis = 1).to_csv('synthetic_data.csv')
```

```
[7] new_data = pd.read_csv('synthetic_data.csv')
    new_data.head()
```

	Unnamed: 0	Cycle_Index	Discharge Time (s)	Decrement 3.6-3.4V (s)	Max. Voltage Dischar. (V)	Min. Voltage Charg. (V)	Time at 4.15V (s)	Time constant current (s)	Charging time (s)	Total time (s)	RUL
0	0	1.0	2743.090913	22877.077465	4.099	3.342	1939.558659	5123.756668	5.980000	420816.340484	1133.0
1	1	2.0	4435.766699	7864.520662	4.020	3.452	8454.935541	17190.176573	20440.712717	490413.959720	1095.0
2	2	11.0	14689.776232	34282.876636	4.046	3.431	13212.530393	19437.151538	39263.680998	848087.093549	1094.0
3	3	14.0	1557.115624	9494.497927	4.023	3.399	1159.464311	461.205452	5.980000	600542.093339	1092.0
4	4	19.0	3478.394719	5687.739733	4.023	3.465	3630.002034	9064.636267	13509.543014	458481.359971	1089.0

Figure 14 Saving and Loading the Synthetic Data

Explanation:

- The synthetic data is saved to ‘synthetic_data.csv’.

It is then reloaded into a dataframe for further analysis.

3.3. Correlation between Synthetic Data with real-world Datasets

In this chapter, we compare the correlation structures of the real-world dataset and the synthetic dataset. This comparison is crucial to evaluate how well the synthetic data mirrors the underlying patterns and relationships of the real-world data. By analyzing correlation matrices and calculating the Frobenius norm, we quantify the similarity between these datasets. This analysis helps in validating the synthetic data generation process and ensuring its reliability for further model training and evaluation.

The relationship between synthetic data and real-world datasets is explored across various dimensions in the reviewed papers. (Rajendran et al., 2023) discusses the importance of understanding and reducing the domain gap between synthetic and real-world data to improve the performance of perception algorithms, highlighting a case where synthetic data optimized for specific classes resulted in a performance boost of over 18%.

(Jenkins, 2023) emphasizes the utility of synthetic data in public policy, enabling faster research while ensuring privacy and security, especially when real-world datasets are limited. (Khadka et al., 2023) introduced a novel synthetic data generation method using combinatorial testing and variational autoencoders, achieving performance parity with models trained on real-world data. (Spell et al., 2023) focus on improving the realism of synthetic IR data by compositing real and synthetic images, thereby enhancing the training efficacy of CNNs in infrared detection and classification tasks.

3.3.1. Correlation Analysis

3.3.1.1. Real-World Dataset Correlation Matrix

The real-world dataset correlation matrix is presented in table 5.

Table 5 Real-World Dataset Correlation Matrix

	Cycle_Index	Discharge Time (s)	Decrement 3.6-3.4V (s)	Max. Voltage Dischar. (V)	Min. Voltage Charg. (V)	Time at 4.15V (s)	Time constant current (s)	Charging time (s)	Total time (s)	RUL
Cycle_Index	1.000	-0.012	-0.012	-0.783	0.760	-0.176	-0.041	-0.018	-0.027	-0.999
Discharge Time (s)	-0.012	1.000	0.445	0.349	-0.239	0.777	0.913	0.936	0.970	0.012
Decrement 3.6-3.4V (s)	-0.012	0.445	1.000	0.199	-0.133	0.477	0.343	0.372	0.438	0.012
Max. Voltage Dischar. (V)	-0.783	0.349	0.199	1.000	-0.718	0.446	0.287	0.269	0.357	0.783
Min. Voltage Charg. (V)	0.760	-0.239	-0.133	-0.718	1.000	-0.344	-0.217	-0.200	-0.167	-0.760

	Cycle_Index	Discharge Time (s)	Decrement 3.6-3.4V (s)	Max. Voltage Dischar. (V)	Min. Voltage Charg. (V)	Time at 4.15V (s)	Time constant current (s)	Charging time (s)	Total time (s)	RUL
Time at 4.15V (s)	-0.176	0.777	0.477	0.446	-0.344	1.000	0.569	0.677	0.769	0.177
Time constant current (s)	-0.041	0.913	0.343	0.287	-0.217	0.569	1.000	0.954	0.882	0.041
Charging time (s)	-0.018	0.936	0.372	0.269	-0.200	0.677	0.954	1.000	0.909	0.018
Total time (s)	-0.027	0.970	0.438	0.357	-0.167	0.769	0.882	0.909	1.000	0.027
RUL	-0.999	0.012	0.012	0.783	-0.760	0.177	0.041	0.018	0.027	1.000

3.3.1.2. Synthetic Dataset Correlation Matrix

The correlation matrix for the synthetic dataset is presented in table 6.

Table 6 Synthetic Dataset Correlation Matrix

	Cycle_Index	Discharge Time (s)	Decrement 3.6-3.4V (s)	Max. Voltage Dischar. (V)	Min. Voltage Charg. (V)	Time at 4.15V (s)	Time constant current (s)	Charging time (s)	Total time (s)	RUL
Cycle_Index	1.000	-0.236	0.025	-0.823	0.766	-0.248	-0.161	-0.064	-0.139	-0.999
Discharge Time (s)	-0.236	1.000	0.275	0.577	-0.384	0.914	0.883	0.839	0.626	0.236
Decrement 3.6-3.4V (s)	0.025	0.275	1.000	0.098	-0.042	0.287	0.283	0.296	0.219	-0.024
Max. Voltage Dischar. (V)	-0.823	0.577	0.098	1.000	-0.747	0.580	0.495	0.414	0.471	0.824
Min. Voltage Charg. (V)	0.766	-0.384	-0.042	-0.747	1.000	-0.410	-0.340	-0.259	-0.108	-0.767
Time at 4.15V (s)	-0.248	0.914	0.287	0.580	-0.410	1.000	0.870	0.835	0.655	0.248

	Cycle_Index	Discharge Time (s)	Decrement 3.6-3.4V (s)	Max. Voltage Dischar. (V)	Min. Voltage Charg. (V)	Time at 4.15V (s)	Time constant current (s)	Charging time (s)	Total time (s)	RUL
Time constant current (s)	-0.161	0.883	0.283	0.495	-0.340	0.870	1.000	0.959	0.675	0.161
Charging time (s)	-0.064	0.839	0.296	0.414	-0.259	0.835	0.959	1.000	0.704	0.064
Total time (s)	-0.139	0.626	0.219	0.471	-0.108	0.655	0.675	0.704	1.000	0.140
RUL	-0.999	0.236	-0.024	0.824	-0.767	0.248	0.161	0.064	0.140	1.000

3.3.2. Frobenius norm

The Frobenius norm is calculated to measure the difference between the real and synthetic correlation matrices:

Frobenius Norm=1.321 Frobenius Norm=1.321

This norm provides a single value quantifying the overall discrepancy between the two correlation matrices.

3.4. Detailed Analysis

3.4.1. Maximum and Minimum Values

In the real-world dataset correlation matrix:

- The maximum value is 1.000 (self-correlation of features).
- The minimum value is -0.999 (correlation between Cycle_Index and RUL).

In the synthetic dataset correlation matrix:

- The maximum value is also 1.000 (self-correlation of features).
- The minimum value is -0.999 (correlation between Cycle_Index and RUL).

3.4.2. Standard Deviation Variations

The spread of correlation values provides insight into the relationships between different features:

- In the real dataset, we observe high correlations such as between Discharge Time (s) and Total time (s) (0.970), and between Cycle_Index and RUL (-0.999).
- The synthetic dataset similarly shows high correlations, like between Discharge Time (s) and Time at 4.15V (s) (0.914), and between Cycle_Index and RUL (-0.999).

These similar spreads indicate that the synthetic data successfully captures the significant relationships present in the real dataset.

3.4.3. Missing Values

There are no missing values in either the real or synthetic correlation matrices, ensuring that all feature relationships are described.

3.4.4. Key Observations

• Cycle_Index and RUL: Both datasets show a strong negative correlation between Cycle_Index and RUL (-0.999 for real and -0.999 for synthetic). This indicates a similar trend in the synthetic data, capturing the relationship between these variables accurately.

• Discharge Time (s) and RUL: The real dataset shows a very weak positive correlation (0.012), while the synthetic dataset shows a stronger positive correlation (0.236). This difference highlights an area where the synthetic data may introduce slightly different relationships.

• Charging Time (s) and Total Time (s): Both datasets show a strong positive correlation between these variables (0.909 for real and 0.704 for synthetic), indicating that the synthetic data maintains the integrity of this relationship.

• Consistency of RUL: Ensuring that RUL values decrease within each DeviceID group across cycles is critical for maintaining the integrity of our analytical insights. Both the real and synthetic datasets adhere to this principle, as demonstrated by the consistently strong negative correlation between Cycle_Index and RUL.

Chapter 4 Exploratory Data Analysis (EDA)

This chapter is based on the exploratory data analysis for the given dataset. Exploratory Data Analysis (EDA) is a primary step in the data analysis method, where we use various methods to précis the main features of the data. We use EDA to expose patterns, spot anomalies, test hypotheses, and check traditions with the help of statistical illustrations and other data visualization approaches. Using this method, we can better recognize the fundamental structure of the data, classify important variables, and detect outliers or errors. EDA helps us to make informed decisions about further investigation, modelling approaches, and data

preprocessing phases. It provides a solid groundwork for data modelling by ensuring we know the data's behavior and characteristics before applying more composite systems.

4.1. Implementation on Dataset

Below are samples and results of various EDA operations on a given dataset. We also have some examples of codes for the given data.

Figure 15 shows the header information of the data having various columns and features such as cycle-index, discharge time, decrement, max, min voltage, and RUL etc. Using the EDA method, we can apply header code and get these results to check what is the behavior of the data.

```
[11] import pandas as pd

    # Assuming 'synthetic_data' is your DataFrame
    # Define the start date
    start_date = pd.Timestamp('2023-01-01')

    # Generate a date range
    date_range = pd.date_range(start=start_date, periods=len(synthetic_data), freq='D')

    # Add the date range as a new column to the DataFrame
    synthetic_data['Timestamp'] = date_range
    synthetic_data
```

	Unnamed: 0	Cycle_Index	Discharge Time (s)	Decrement 3.6-3.4V (s)	Max. Voltage Dischar. (V)	Min. Voltage Charg. (V)	Time at 4.15V (s)	Time constant current (s)	Charging time (s)	Total time (s)	RUL	Timestamp
0	0	1.0	2743.090913	22877.077465	4.099	3.342	1939.558659	5123.756668	5.980000	420816.340484	1133.0	2023-01-01
1	1	2.0	4435.766699	7864.520662	4.020	3.452	8454.935541	17190.176573	20440.712717	490413.959720	1095.0	2023-01-02
2	2	11.0	14689.776232	34282.876636	4.046	3.431	13212.530393	19437.151538	39263.680998	848087.093549	1094.0	2023-01-03
3	3	14.0	1557.115624	9494.497927	4.023	3.399	1159.464311	461.205452	5.980000	600542.093339	1092.0	2023-01-04
4	4	19.0	3478.394719	5687.739733	4.023	3.465	3630.002034	9064.636267	13509.543014	458481.359971	1089.0	2023-01-05
...	...	...	...	...	...	...	...	...	...	...	...	...
3025	3025	1077.0	227.231498	1117.330655	3.745	3.760	-113.584000	5.980000	5.980000	332798.845431	18.0	2031-04-14
3026	3026	1083.0	10587.448518	43115.434264	3.842	3.732	17410.999936	28987.433898	45588.695575	759604.696932	14.0	2031-04-15
3027	3027	1096.0	719.983899	25695.446385	3.741	3.740	-113.584000	5.980000	5693.826296	362683.572260	11.0	2031-04-16
3028	3028	1099.0	4568.416236	28249.771627	3.804	3.687	6307.775096	23794.495574	31109.238923	835986.270423	8.0	2031-04-17
3029	3029	1100.0	6089.571972	21654.857123	3.810	3.765	11385.532230	31510.326857	44743.882146	845614.971856	0.0	2031-04-18

3030 rows × 12 columns

Figure 15 Header information.

After the visualization of the header, the next step is to check the details of every feature, which type of data it has and which type of data is going to be process; for this, we use the information view of the data using the features view operation, in Figure 2 we apply 3-4 features which depend on every feature set of the data.

4.2. Feature Representation

We have dependent features such as cycle index and charge time, decrement and max voltage. However, figure 16 shows the count plot of the dataset; we target our main feature

set, RUL and check the count of the dataset. We select the bar plot to show the data and apply various colour schemes to understand it more accurately.

```
(                          Cycle_Index  Discharge Time (s)  \
 Cycle_Index                  1.000000           -0.011925
 Discharge Time (s)          -0.011925            1.000000
 Decrement 3.6-3.4V (s)      -0.012090            0.444984
 Max. Voltage Dischar. (V)   -0.782859            0.349161
 Min. Voltage Charg. (V)      0.759850           -0.239410
 Time at 4.15V (s)           -0.176486            0.776720
 Time constant current (s)   -0.040622            0.912536
 Charging time (s)           -0.018305            0.935544
 Total time (s)              -0.026809            0.969673
 RUL                         -0.999756            0.011957

                           Decrement 3.6-3.4V (s)  Max. Voltage Dischar. (V)  \
 Cycle_Index                            -0.012090                  -0.782859
 Discharge Time (s)                      0.444984                   0.349161
 Decrement 3.6-3.4V (s)                  1.000000                   0.199470
 Max. Voltage Dischar. (V)               0.199470                   1.000000
 Min. Voltage Charg. (V)                -0.132765                  -0.718182
 Time at 4.15V (s)                       0.477091                   0.446070
 Time constant current (s)               0.343228                   0.286886
 Charging time (s)                       0.371610                   0.269009
 Total time (s)                          0.438041                   0.357224
 RUL                                     0.011998                   0.782800

                           Min. Voltage Charg. (V)  Time at 4.15V (s)  \
```

Figure 16 View of features with respect to other features

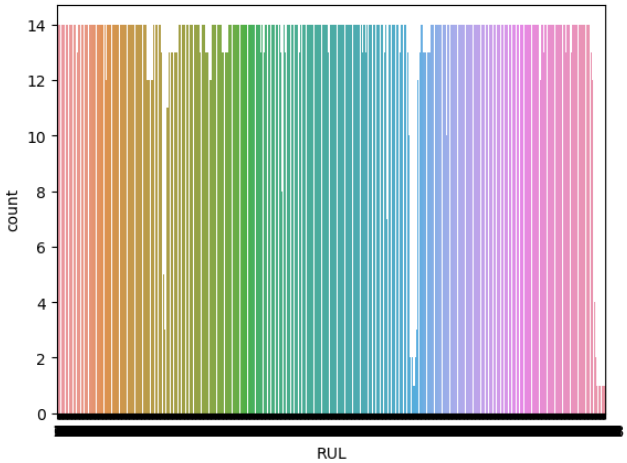


Figure 17 Count plot

After the count plot and features discerption, we need to check the original and real data in term of features and concerning other feature sets, we check the real data in terms of discharge time vs cycle. Figure 17 shows the real data for discharge time vs cycle.

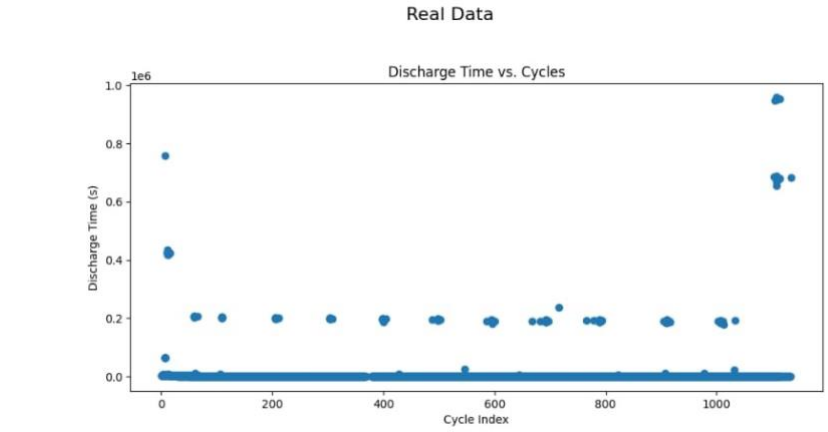


Figure 18 The real data in terms of RUL vs cycle.

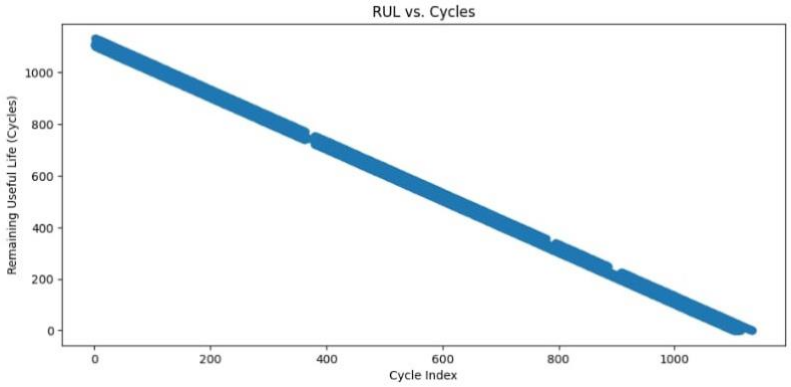


Figure 19 The real data in terms of Total time vs RUL.

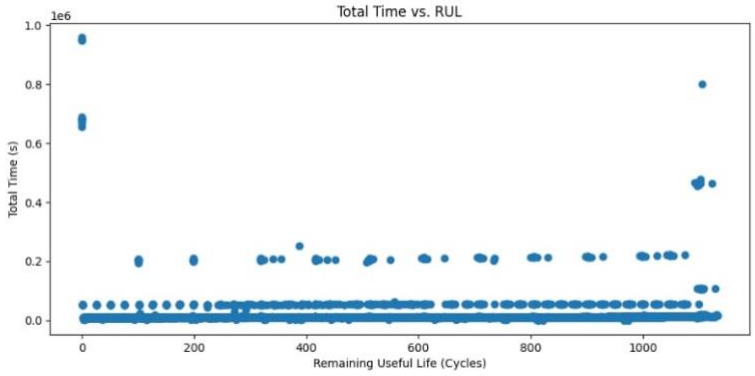


Figure 20 The real data in terms of Total time vs RUL.

The next step is to show the relation of the data in terms of synthetic data; for this, figure 7 shows the synthetic data description of time vs cycles, figure 19 shows RUL vs cycle and Figure 20 shows RUL vs total time.

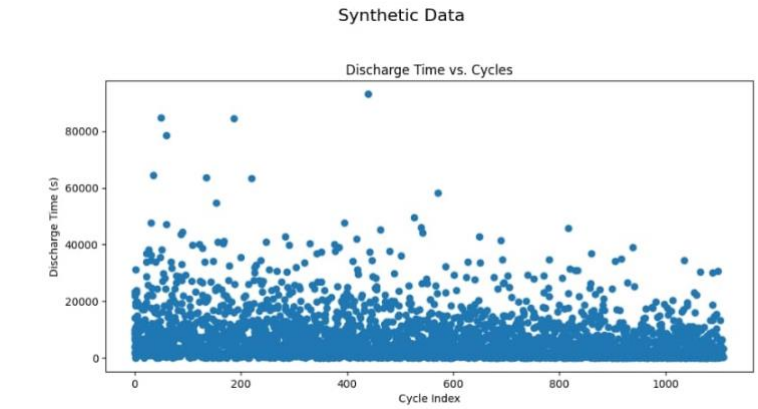


Figure 21 The Synthetic data in term of Time vs Cycles.

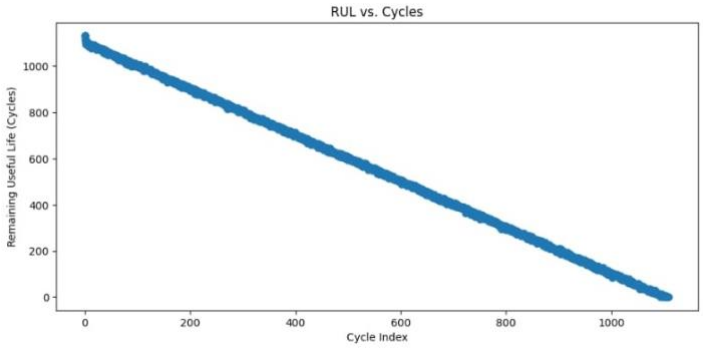


Figure 22 The Synthetic data in terms of RUL **vs Cycles.**

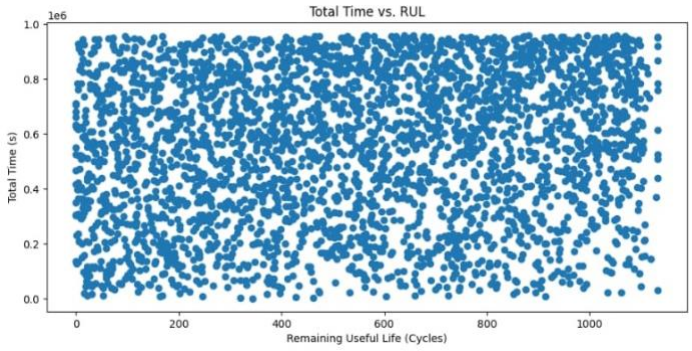


Figure 23 The Synthetic data in terms of Total time vs RUL.

After visualizing basic EDA, we were required to find a correlation between all variables; the correlation matrix illustrates correlation coefficients between variables. Each cell in the table denotes the correlation between two variables from -1 to 1. We use the correlation matrix to recognize the strength and direction of linear relationships between variables. This

matrix is valuable because it helps identify highly correlated variables, which can be essential for feature selection and multicollinearity recognition.

By examining the correlation matrix, we can streamline our models by removing redundant features, improving model performance, and gaining deeper perceptions of the data's relationships. It is an essential tool in the preprocessing stage. We apply the following Code and extract the co-relation matrix. Figure 24 shows the complete picture of the correlation matrix.

```
[10] import numpy as np

     # Calculate correlation matrices for both datasets
     correlation_matrix_real = real_data.drop(['DeviceID'],axis = 1).iloc[:, 1:].corr()  # Exclude the Unnamed: 0 column
     correlation_matrix_synthetic = synthetic_data.iloc[:, 1:].corr()

     correlation_matrix_real_display = correlation_matrix_real
     correlation_matrix_synthetic_display = correlation_matrix_synthetic

     # Recalculate the Frobenius norm
     frobenius_norm_recalculation = np.linalg.norm(correlation_matrix_real - correlation_matrix_synthetic)

     (correlation_matrix_real_display, correlation_matrix_synthetic_display, frobenius_norm_recalculation)
```

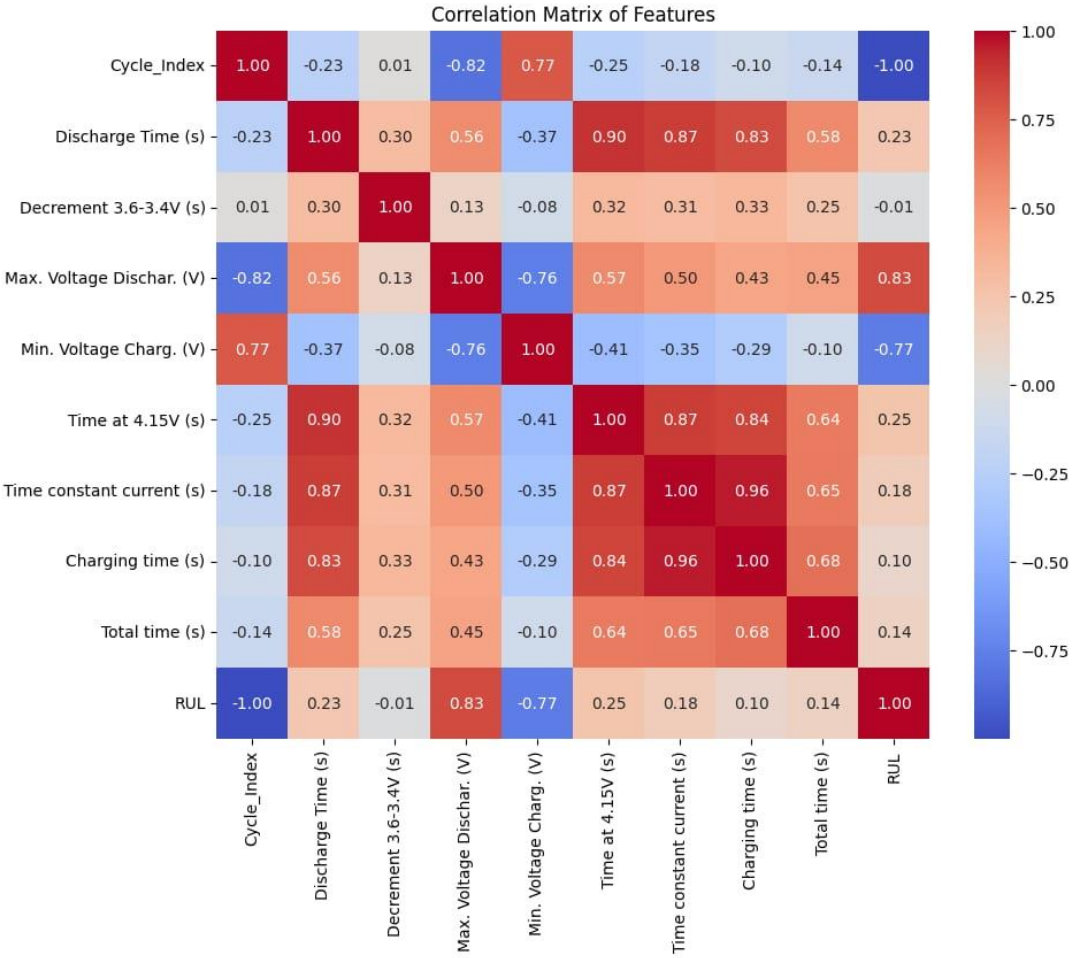
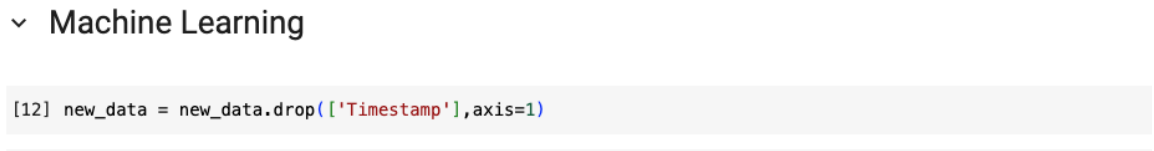


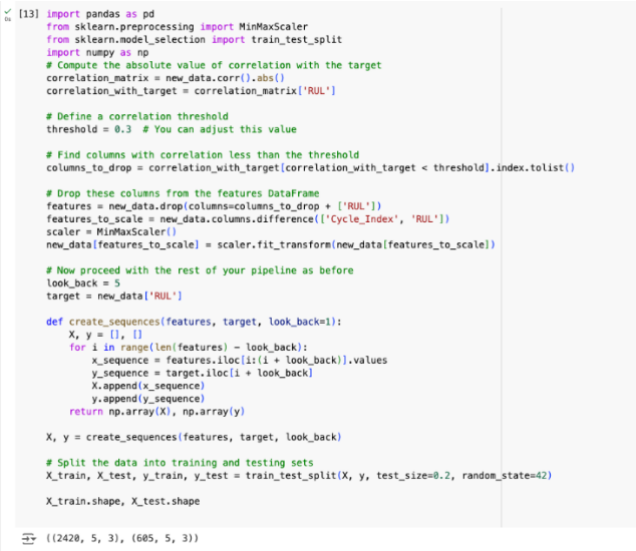
Figure 24 Overview of the correlation matrix.

Chapter 5 Feature Engineering, Machine Learning, and Conclusion

5.1. Feature Engineering:

Feature engineering is one of the primary parts of every data analysis project; in this project, we applied a basic preprocessing method and after this, we extracted useful features; we used the Pandas library for preprocessing and model selection, and after preprocessing, we split the data into two parts such as x and y. Figures 11 and 12 show the detailed Code of data preprocessing and installation of various libraries.





```
[13] import pandas as pd
     from sklearn.preprocessing import MinMaxScaler
     from sklearn.model_selection import train_test_split
     import numpy as np
     # Compute the absolute value of correlation with the target
     correlation_matrix = new_data.corr().abs()
     correlation_with_target = correlation_matrix['RUL']

     # Define a correlation threshold
     threshold = 0.3  # You can adjust this value

     # Find columns with correlation less than the threshold
     columns_to_drop = correlation_with_target[correlation_with_target < threshold].index.tolist()

     # Drop these columns from the features DataFrame
     features = new_data.drop(columns=columns_to_drop + ['RUL'])
     features_to_scale = new_data.columns.difference(['Cycle_Index', 'RUL'])
     scaler = MinMaxScaler()
     new_data[features_to_scale] = scaler.fit_transform(new_data[features_to_scale])

     # Now proceed with the rest of your pipeline as before
     look_back = 5
     target = new_data['RUL']

     def create_sequences(features, target, look_back=1):
         X, y = [], []
         for i in range(len(features) - look_back):
             x_sequence = features.iloc[i:(i + look_back)].values
             y_sequence = target.iloc[i + look_back]
             X.append(x_sequence)
             y.append(y_sequence)
         return np.array(X), np.array(y)

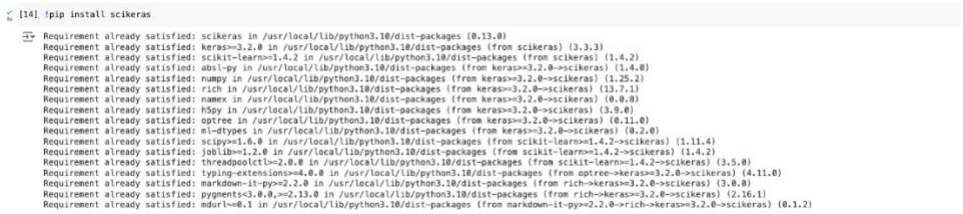
     X, y = create_sequences(features, target, look_back)

     # Split the data into training and testing sets
     X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

     X_train.shape, X_test.shape

→ ((2420, 5, 3), (605, 5, 3))
```

Figure 25 Overview of the correlation matrix.



```
[14] !pip install scikeras

→ Requirement already satisfied: scikeras in /usr/local/lib/python3.10/dist-packages (0.13.0)
  Requirement already satisfied: keras>=3.2.0 in /usr/local/lib/python3.10/dist-packages (from scikeras) (3.3.3)
  Requirement already satisfied: scikit-learn>=1.4.2 in /usr/local/lib/python3.10/dist-packages (from scikeras) (1.4.2)
  Requirement already satisfied: absl-py in /usr/local/lib/python3.10/dist-packages (from keras>=3.2.0->scikeras) (1.4.0)
  Requirement already satisfied: numpy in /usr/local/lib/python3.10/dist-packages (from keras>=3.2.0->scikeras) (1.25.2)
  Requirement already satisfied: rich in /usr/local/lib/python3.10/dist-packages (from keras>=3.2.0->scikeras) (13.7.1)
  Requirement already satisfied: namex in /usr/local/lib/python3.10/dist-packages (from keras>=3.2.0->scikeras) (0.0.8)
  Requirement already satisfied: h5py in /usr/local/lib/python3.10/dist-packages (from keras>=3.2.0->scikeras) (3.9.0)
  Requirement already satisfied: optree in /usr/local/lib/python3.10/dist-packages (from keras>=3.2.0->scikeras) (0.11.0)
  Requirement already satisfied: ml-dtypes in /usr/local/lib/python3.10/dist-packages (from keras>=3.2.0->scikeras) (0.2.0)
  Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.10/dist-packages (from scikit-learn>=1.4.2->scikeras) (1.11.4)
  Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.10/dist-packages (from scikit-learn>=1.4.2->scikeras) (1.4.2)
  Requirement already satisfied: threadpoolctl>=2.0.0 in /usr/local/lib/python3.10/dist-packages (from scikit-learn>=1.4.2->scikeras) (3.5.0)
  Requirement already satisfied: typing-extensions>=4.0.0 in /usr/local/lib/python3.10/dist-packages (from optree->keras>=3.2.0->scikeras) (4.11.0)
  Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.10/dist-packages (from rich->keras>=3.2.0->scikeras) (3.0.0)
  Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.10/dist-packages (from rich->keras>=3.2.0->scikeras) (2.16.1)
  Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.10/dist-packages (from markdown-it-py>=2.2.0->rich->keras>=3.2.0->scikeras) (0.1.2)
```

Figure 26 Installation of library

5.2. Machine Learning:

Machine learning is an advanced method for data analysis and predicting given data. We applied the LSTM technique as a machine learning model in this project. LSTM, which stands for Long Short-Term Memory, is a neural network that handles chronological and time-series data. Distinct traditional neural networks, LSTMs can learn and remember important patterns over long categorizations and assemble them idyllic for language modelling, such as stock price prediction and speech recognition tasks. In addition, we utilized XGBoost library for gradient boosting framework.

5.2.1. LSTM

We use LSTM as it effectively achieves the needs and relationships in data that clarify over time. The data required for LSTM models classically contains time series, sequences, or any

data where the information order is critical. For example, we might use stock prices, climate data, and sequences of words. The assistance of machine learning includes the ability to program complex decision-making methods, expose hidden patterns in data, and expand over time with more data.

Limitations: ML models require large volumes of data to perform fit, and they occasionally be seen as "black boxes" because their decision-making processes aren't always transparent. LSTMs also have their specific limitations. They can be computationally exhaustive and require significant resources to train. LSTMs get stuck with long arrangements when insufficient data accurately captures the fundamental patterns. Despite these challenges, LSTMs remain powerful tools for dealing with sequential data, providing valuable visions and accurate estimates in many real-world applications. Figure 27 shows the Code of LSTM.



```
[16] from keras.models import Sequential
     from keras.layers import LSTM, Dense
     from keras.optimizers import Adam

     # Build the LSTM model
     lstm_model = Sequential([
         LSTM(50, input_shape=(X_train.shape[1], X_train.shape[2]), return_sequences=True),
         LSTM(50, return_sequences=False),
         Dense(50, activation='relu'),
         Dense(1)
     ])

     # Compile the model
     lstm_model.compile(loss='mean_squared_error', optimizer=Adam(learning_rate=0.001))

     # Train the model
     lstm_history = lstm_model.fit(X_train, y_train, epochs=50, batch_size=32, validation_split=0.2, verbose=1)
```

Figure 27 Code of LSTM

Figure 28 shows the detailed Code of LSTM and its operation, various libraries calls, Lstm unit, batch size, optimization and epochs.

```
[15] from tensorflow.keras.models import Sequential
     from tensorflow.keras.layers import LSTM, Dense
     from tensorflow.keras.optimizers import Adam
     import numpy as np

     # List of hyperparameters to try
     lstm_units_options = [20, 50, 100]
     optimizer_lr_options = [0.001, 0.01, 0.1]
     batch_size_options = [16, 32, 64]
     epochs_options = [5, 10, 30]

     # Record the best score and corresponding hyperparameters
     best_score = np.inf
     best_params = {}

     for lstm_units in lstm_units_options:
         for optimizer_lr in optimizer_lr_options:
             for batch_size in batch_size_options:
                 for epochs in epochs_options:
                     # Create the model with the current set of hyperparameters
                     model = Sequential([
                         LSTM(lstm_units, input_shape=(X_train.shape[1], X_train.shape[2]), return_sequences=True),
                         LSTM(lstm_units, return_sequences=False),
                         Dense(50, activation='relu'),
                         Dense(1)
                     ])
                     print(1)
                     model.compile(loss='mean_squared_error', optimizer=Adam(learning_rate=optimizer_lr))

                     # Fit the model
                     history = model.fit(X_train, y_train, epochs=epochs, batch_size=batch_size, validation_split=0.2, verbose=0)

                     # Evaluate the model
                     val_loss = np.min(history.history['val_loss'])  # Get the best validation loss achieved

                     # Update the best parameters if the current model is better
                     if val_loss < best_score:
                         best_score = val_loss
                         best_params = {
                             'lstm_units': lstm_units,
                             'optimizer_lr': optimizer_lr,
                             'batch_size': batch_size,
                             'epochs': epochs
                         }

     # Print the best model's parameters and its score
     print("Best Validation Loss: {:.4f}".format(best_score))
     print("Best Parameters:")
     print(best_params)
```

Figure 28 Detailed code of LSTM

Figure 29 shows the Code for mean square error, mean absolute error and r2 error.

```
[18] from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

     # Make predictions
     y_pred_lstm = lstm_model.predict(X_test)
     n=len(X)
     p=X.shape[1]
     # Calculate metrics
     rmse_lstm = np.sqrt(mean_squared_error(y_test, y_pred_lstm))
     mae_lstm = mean_absolute_error(y_test, y_pred_lstm)
     r2_lstm = r2_score(y_test, y_pred_lstm)
     r2_adj_lstm = 1 - ((1-r2_lstm) * (n-1) / (n-p-1))  # Adjusted R² assuming p (number of features) is the number of inputs to the model

     print(f"RMSE: {rmse_lstm}, MAE: {mae_lstm}, R²: {r2_lstm}, Adjusted R²: {r2_adj_lstm}")

19/19 ━━━━━━━━━━━━━━━━━━━━ 1s 21ms/step
RMSE: 46.54524381931605, MAE: 12.915217577721462, R²: 0.9790287031802966, Adjusted R²: 0.9789939709894723
```

Figure 29 Code for mean square error, mean absolute error and r2 error.

5.2.2. XGBoost

We designed hyperparameter tuning for an XGBoost regression model using Grid Search with Cross-Validation (CV). The goal is to find the optimal combination of hyperparameters to improve the model's performance on a given dataset. We systematically analyzeddifferent combinations of hyperparameters, the framework aims to find the best configuration that

minimizes the mean squared error, thus improving the model's predictive performance on the given dataset.

The study involves training an XGBoost regression model on a dataset with flattened features and evaluating its performance via various metrics, such as RMSE, MAE, R^2, and adjusted R^2. The aim is to show the effectiveness of the model in predicting the target variable and to provide insights into its accuracy and reliability.

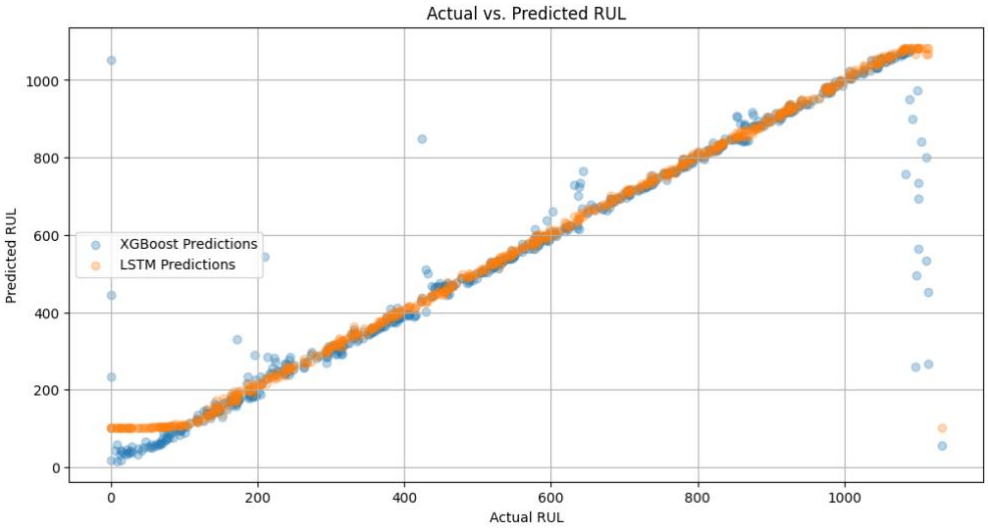


Figure 30 Comparison of XGBoost predictions and LSTM predictions

The figure 30 shows visually compares the performance of two predictive models, XGBoost and LSTM, in estimating the RUL of components. The result comparison shows that while both models perform well in most cases, there are areas where predictions can be further improved, especially for higher RUL values. This analysis complements the quantitative evaluation metrics, providing a comprehensive understanding of model performance/

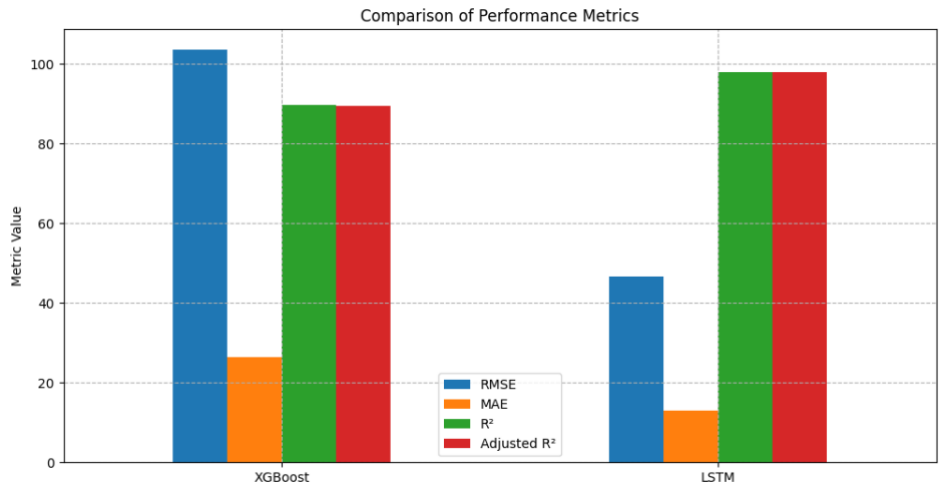


Figure 31 Comparison of Performance Metrics

Figure 31 shows both models perform well, the LSTM model exhibits superior performance across all evaluation metrics. This suggests that LSTM is better suited for predicting the RUL of components in this dataset. The comparison provides a better understanding of how each model performs in different aspects of predictive accuracy and explanatory power.

5.3. Conclusion

Initially, the research began by examining real battery dataset to understand its structure and properties. This step allowed us to capture key information about the dataset's columns, their types, and distributions. With the metadata in hand, we configured a Gaussian Copula Synthesizer, a powerful tool for generating synthetic data while preserving statistical dependencies. We specified distributions for numerical columns like discharge time and total time, ensuring that the synthetic data reflects the variability observed in our real dataset. Second, synthesizer was trained on real battery data. This process involved teaching the synthesizer to understand the underlying statistical patterns and dependencies present in the data. Using the trained synthesizer, we generated a synthetic dataset containing 100,000 rows. This synthetic data is a simulated representation of battery performance based on the statistical patterns learned from our real dataset.

To ensure the quality and relevance of the synthetic data, we performed several post-processing steps. This included sorting the data by device ID and RUL and filtering out rows with positive decrement values, ensuring consistency with our real data. Finally, we took

steps to ensure that the synthetic data maintains the same characteristics as our real dataset. We verified that RUL values decrease within each device ID group across cycles, maintaining the integrity of our analytical insights.

References

Afrasiabi, M., Mohammadi, M., Rastegar, M., & Kargarian, A. (2019). Multi-agent microgrid energy management based on deep learning forecaster. Energy, 186, 115873.

Armand, M., Axmann, P., Bresser, D., Copley, M., Edström, K., Ekberg, C., ... & Zhang, H. (2020). Lithium-ion batteries–Current state of the art and anticipated developments. *Journal of Power Sources*, *479*, 228708.

Atalay, S., Sheikh, M., Mariani, A., Merla, Y., Bower, E., & Widanage, W. D. (2020). Theory of battery ageing in a lithium-ion battery: Capacity fade, nonlinear ageing and lifetime prediction. Journal of Power Sources, 478, 229026.

Bi, Y., Yin, Y., & Choe, S. Y. (2020). Online state of health and aging parameter estimation using a physics-based life model with a particle filter. Journal of Power Sources, 476, 228655.

Bermúdez, J. D., Achanccaray, P., Sanches, I. D., Cue, L., Happ, P., & Feitosa, R. Q. (2017, November). Evaluation of recurrent neural networks for crop recognition from multitemporal remote sensing images. In *Anais do XXVII Congresso Brasileiro de Cartografia* (pp. 800-804).

Cai, Y., Yang, L., Deng, Z., Zhao, X., & Deng, H. (2017, September). Prediction of lithium-ion battery remaining useful life based on hybrid data-driven method with optimized parameter. In 2017 2nd International Conference on Power and Renewable Energy (ICPRE) (pp. 1-6). IEEE.

Che, Y., Deng, Z., Lin, X., Hu, L., & Hu, X. (2021). Predictive battery health management with transfer learning and online model correction. IEEE Transactions on Vehicular Technology, 70(2), 1269-1277.

Chehade, A. A., & Hussein, A. A. (2020). A collaborative Gaussian process regression model for transfer learning of capacity trends between li-ion battery cells. IEEE Transactions on Vehicular Technology, 69(9), 9542-9552.

Chen, L., An, J., Wang, H., Zhang, M., & Pan, H. (2020). Remaining useful life prediction for lithium-ion battery by combining an improved particle filter with sliding-window gray model. Energy Reports, 6, 2086-2093.

Chen, Zewang, et al. "Lithium-ion batteries remaining useful life prediction based on BLS-RVM." Energy 234 (2021): 121269.

Cheng, G., Wang, X., & He, Y. (2021). Remaining useful life and state of health prediction for lithium batteries based on empirical mode decomposition and a long and short memory neural network. Energy, 232, 121022.

Chu, A., Allam, A., Arenas, A. C., Rizzoni, G., & Onori, S. (2020). Stochastic capacity loss and remaining useful life models for lithium-ion batteries in plug-in hybrid electric vehicles. Journal of Power Sources, 478, 228991.

Ciccarelli, M., Corradini, F., Germani, M., Menchi, G., Mostarda, L., Papetti, A., & Piangerelli, M. (2023). SPECTRE: a deep learning network for posture recognition in manufacturing. *Journal of Intelligent Manufacturing*, *34*(8), 3469-3481.

Che, Y., Hu, X., Lin, X., Guo, J., & Teodorescu, R. (2023). Health prognostics for lithium-ion batteries: mechanisms, methods, and prospects. *Energy & Environmental Science*, *16*(2), 338-371.

Cong, X., Zhang, C., Jiang, J., Zhang, W., & Jiang, Y. (2020). A hybrid method for the prediction of the remaining useful life of lithium-ion batteries with accelerated capacity degradation. IEEE Transactions on Vehicular Technology, 69(11), 12775-12785.

Cui, Z., Cui, N., Wang, C., Li, C., & Zhang, C. (2020). A robust online parameter identification method for lithium-ion battery model under asynchronous sampling and noise interference. IEEE Transactions on Industrial Electronics, 68(10), 9550-9560.

D. Liu, W. Xie, S. Lu, Y. Peng, in: 2015 Annu. Reliab. Maintainab. Symp., IEEE, 2015, pp. 1–6

Deng, D. (2015). Li-ion batteries: basics, progress, and challenges. Energy Science & Engineering, 3(5), 385-418.

Devie, A., Baure, G., & Dubarry, M. (2018). Intrinsic variability in the degradation of a batch of commercial 18650 lithium-ion cells. Energies, 11(5), 1031.

Dong, G., Xu, Y., & Wei, Z. (2021). A Hierarchical Approach for Finite-Time H-$\infty $ State-of-Charge Observer and Probabilistic Lifetime Prediction of Lithium-Ion Batteries. IEEE Transactions on Energy Conversion, 37(1), 718-728.

Du, J., Liu, Y., Mo, X., Li, Y., Li, J., Wu, X., & Ouyang, M. (2019). Impact of high-power charging on the durability and safety of lithium batteries used in long-range battery electric vehicles. Applied Energy, 255, 113793.

El Mejdoubi, A., Chaoui, H., Gualous, H., Van Den Bossche, P., Omar, N., & Van Mierlo, J. (2018). Lithium-ion batteries health prognosis considering aging conditions. IEEE Transactions on Power Electronics, 34(7), 6834-6844.

Eleftheroglou, N., Mansouri, S. S., Loutas, T., Karvelis, P., Georgoulas, G., Nikolakopoulos, G., & Zarouchas, D. (2019). Intelligent data-driven prognostic methodologies for the real-time remaining useful life until the end-of-discharge estimation of the Lithium-Polymer batteries of unmanned aerial vehicles with uncertainty quantification. Applied Energy, 254, 113677.

Endres, M., Mannarapotta Venugopal, A., & Tran, T. S. (2022, August). Synthetic data generation: a comparative study. In Proceedings of the 26th International Database Engineered Applications Symposium (pp. 94-102).

Fan, J., Fan, J., Liu, F., Qu, J., & Li, R. (2019). A novel machine learning method based approach for Li-ion battery prognostic and health management. Ieee Access, 7, 160043-160061.

Fan, J., Fan, J., Liu, F., Qu, J., & Li, R. (2019). A novel machine learning method based approach for Li-ion battery prognostic and health management. Ieee Access, 7, 160043-160061.

Farmann, A., & Sauer, D. U. (2018). Comparative study of reduced order equivalent circuit models for on-board state-of-available-power prediction of lithium-ion batteries in electric vehicles. Applied Energy, 225, 1102-1122.

Feng, H., & Song, D. (2021). A health indicator extraction based on surface temperature for lithium-ion batteries remaining useful life prediction. Journal of Energy Storage, 34, 102118.

Feng, T., Zhao, Y., Shi, H., Li, H., Zhang, X., Wang, G., ... & Cherry, S. R. (2019, October). The effects of delay on the input function for early dynamics in total body parametric imaging. In 2019 IEEE Nuclear Science Symposium and Medical Imaging Conference (NSS/MIC) (pp. 1-6). IEEE.

Feng, Y., & Dong, Z. (2020). Optimal energy management with balanced fuel economy and battery life for large hybrid electric mining truck. Journal of Power Sources, 454, 227948.

Finegan, D. P., Zhu, J., Feng, X., Keyser, M., Ulmefors, M., Li, W., ... & Cooper, S. J. (2021). The application of data-driven methods and physics-based learning for improving battery safety. Joule, 5(2), 316-329.

Gou, B., Xu, Y., & Feng, X. (2020). State-of-health estimation and remaining-useful-life prediction for lithium-ion battery using a hybrid data-driven method. *IEEE Transactions on Vehicular Technology*, *69*(10), 10854-10867.

Gu, X., See, K. W., Wang, Y., Zhao, L., & Pu, W. (2021). The sliding window and SHAP theory—an improved system with a long short-term memory network model for state of charge prediction in electric vehicle application. *Energies*, *14*(12), 3692.

Guha, A., & Patra, A. (2018). Online estimation of the electrochemical impedance spectrum and remaining useful life of lithium-ion batteries. IEEE Transactions on Instrumentation and Measurement, 67(8), 1836-1849.

Hafiz, F., Awal, M. A., de Queiroz, A. R., & Husain, I. (2020). Real-time stochastic optimization of energy storage management using deep learning-based forecasts for residential PV applications. IEEE Transactions on Industry Applications, 56(3), 2216-2226.

Han, X., He, H., Wu, J., Peng, J., & Li, Y. (2019). Energy management based on reinforcement learning with double deep Q-learning for a hybrid electric tracked vehicle. Applied Energy, 254, 113708.

Hao, S., Han, W., Jiang, T., Li, Y., Wu, H., Zhong, C., ... & Tang, H. (2024). Synthetic Data in AI: Challenges, Applications, and Ethical Implications. arXiv preprint arXiv:2401.01629.

He, K., Su, Z., Tian, X., Yu, H., & Luo, M. (2022). RUL prediction of wind turbine gearbox bearings based on self-calibration temporal convolutional network. IEEE Transactions on Instrumentation and Measurement, 71, 1-12.

Hesse, H. C., Schimpe, M., Kucevic, D., & Jossen, A. (2017). Lithium-ion battery storage for the grid—A review of stationary battery storage system design tailored for applications in modern power grids. Energies, 10(12), 2107.

Hong, J., Lee, D., Jeong, E. R., & Yi, Y. (2020). Towards the swift prediction of the remaining useful life of lithium-ion batteries with end-to-end deep learning. Applied energy, 278, 115646.

Hong, J., Wang, Z., Chen, W., & Yao, Y. (2019). Synchronous multi-parameter prediction of battery systems on electric vehicles using long short-term memory networks. Applied Energy, 254, 113648.

Howard, R. L. (2020). Lagrange-Based Options for Relay Satellites to Eliminate Earth-Mars Communications Outages During Solar Superior Conjunctions. In *ASCEND 2020* (p. 4005).

Jenkins, K. (2023). Synthetic Data and Public Policy: supporting real-world policymakers with algorithmically generated data. *Policy Quarterly*, *19*(2), 29-39.

Ji, Y., Chen, Z., Shen, Y., Yang, K., Wang, Y., & Cui, J. (2021). An RUL prediction approach for lithium-ion battery based on SADE-MESN. Applied Soft Computing, 104, 107195.

Jiang, J., Ruan, H., Sun, B., Wang, L., Gao, W., & Zhang, W. (2018). A low-temperature internal heating strategy without lifetime reduction for large-size automotive lithium-ion battery pack. Applied Energy, 230, 257-266.

Jiang, Y., Jiang, J., Zhang, C., Zhang, W., Gao, Y., & Mi, C. (2019). A Copula-based battery pack consistency modeling method and its application on the energy utilization efficiency estimation. Energy, 189, 116219.

Jiao, R., Peng, K., & Dong, J. (2020). Remaining useful life prediction of lithium-ion batteries based on conditional variational autoencoders-particle filter. *IEEE Transactions on Instrumentation and Measurement*, *69*(11), 8831-8843.

Jordon, J., Szpruch, L., Houssiau, F., Bottarelli, M., Cherubin, G., Maple, C., ... & Weller, A. (2022). Synthetic Data--what, why and how?. arXiv preprint arXiv:2205.03257.

Khadka, K., Chandrasekaran, J., Lei, Y., Kacker, R. N., & Kuhn, D. R. (2023, April). Synthetic Data Generation Using Combinatorial Testing and Variational Autoencoder. In *2023 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW)* (pp. 228-236). IEEE.

Kumar, R. R., Bharatiraja, C., Udhayakumar, K., Devakirubakaran, S., Sekar, S., & Mihet-Popa, L. (2023). Advances in batteries, battery modeling, battery management system, battery thermal management, SOC, SOH, and charge/discharge characteristics in EV applications. IEEE Access.

Kwak, E., Jeong, S., Kim, J. H., & Oh, K. Y. (2021). Prediction of compression force evolution over degradation for a lithium-ion battery. Journal of Power Sources, 483, 229079.

Lan, T., Jermsittiparsert, K., T. Alrashood, S., Rezaei, M., Al-Ghussain, L., & A. Mohamed, M. (2021). An advanced machine learning based energy management of renewable microgrids considering hybrid electric vehicles' charging demand. Energies, 14(3), 569.

Lee, C., Jo, S., Kwon, D., & Pecht, M. G. (2020). Capacity-fading behavior analysis for early detection of unhealthy Li-ion batteries. IEEE Transactions on Industrial Electronics, 68(3), 2659-2666.

Li, H., Wan, Z., & He, H. (2019). Constrained EV charging scheduling based on safe deep reinforcement learning. IEEE Transactions on Smart Grid, 11(3), 2427-2439.

Li, P., Zhang, Z., Xiong, Q., Ding, B., Hou, J., Luo, D., ... & Li, S. (2020). State-of-health estimation and remaining useful life prediction for the lithium-ion battery based on a variant long short term memory neural network. *Journal of power sources*, *459*, 228069.

Li, X., Yu, D., Byg, V. S., & Ioan, S. D. (2023). The development of machine learning-based remaining useful life prediction for lithium-ion batteries. Journal of Energy Chemistry, 82, 103-121.

Li, X., Yuan, C., & Wang, Z. (2020). Multi-time-scale framework for prognostic health condition of lithium battery using modified Gaussian process regression and nonlinear regression. Journal of Power Sources, 467, 228358.

Li, Y., Liu, K., Foley, A. M., Zülke, A., Berecibar, M., Nanini-Maury, E., ... & Hoster, H. E. (2019). Data-driven health estimation and lifetime prediction of lithium-ion batteries: A review. Renewable and sustainable energy reviews, 113, 109254.

Lin, C. P., Cabrera, J., Yang, F., Ling, M. H., Tsui, K. L., & Bae, S. J. (2020). Battery state of health modeling and remaining useful life prediction through time series model. Applied Energy, 275, 115338.

Lin, K., Chen, Y., Liu, Y., & Zhang, B. (2020). Reliability prediction of battery management system for electric vehicles based on accelerated degradation test: A semi-parametric approach. IEEE Transactions on Vehicular Technology, 69(11), 12694-12704.

Lin, T., Wang, H., Guo, X., Wang, P., & Song, L. (2023). A novel prediction network for remaining useful life of rotating machinery. The International Journal of Advanced Manufacturing Technology, 124(11), 4009-4018.

Lipu, M. H., Hannan, M. A., Hussain, A., Hoque, M. M., Ker, P. J., Saad, M. H. M., & Ayob, A. (2018). A review of state of health and remaining useful life estimation methods for lithium-ion battery in electric vehicles: Challenges and recommendations. Journal of cleaner production, 205, 115-133.

Liu, C., Wang, Y., & Chen, Z. (2019). Degradation model and cycle life prediction for lithium-ion battery used in hybrid energy storage system. Energy, 166, 796-806.

Liu, D., Luo, Y., Liu, J., Peng, Y., Guo, L., & Pecht, M. (2014). Lithium-ion battery remaining useful life estimation based on fusion nonlinear degradation AR model and RPF algorithm. Neural Computing and Applications, 25, 557-572.

Liu, K., Ashwin, T. R., Hu, X., Lucu, M., & Widanage, W. D. (2020). An evaluation study of different modelling techniques for calendar ageing prediction of lithium-ion batteries. Renewable and Sustainable Energy Reviews, 131, 110017.

Liu, K., Hu, X., Wei, Z., Li, Y., & Jiang, Y. (2019). Modified Gaussian process regression models for cyclic capacity prediction of lithium-ion batteries. IEEE Transactions on Transportation Electrification, 5(4), 1225-1236.

Liu, K., Shang, Y., Ouyang, Q., & Widanage, W. D. (2020). A data-driven approach with uncertainty quantification for predicting future capacities and remaining useful life of lithium-ion battery. IEEE Transactions on Industrial Electronics, 68(4), 3170-3180.

Liu, Y., Zhao, G., & Peng, X. (2019). Deep learning prognostics for lithium-ion battery based on ensembled long short-term memory networks. IEEE Access, 7, 155130-155142.

Lui, Y. H., Li, M., Downey, A., Shen, S., Nemani, V. P., Ye, H., ... & Hu, C. (2021). Physics-based prognostics of implantable-grade lithium-ion battery for remaining useful life prediction. Journal of Power Sources, 485, 229327.

Ma, G., Zhang, Y., Cheng, C., Zhou, B., Hu, P., & Yuan, Y. (2019). Remaining useful life prediction of lithium-ion batteries based on false nearest neighbors and a hybrid neural network. Applied Energy, 253, 113626.

Ma, Y., Wu, L., Guan, Y., & Peng, Z. (2020). The capacity estimation and cycle life prediction of lithium-ion batteries using a new broad extreme learning machine approach. Journal of Power Sources, 476, 228581.

Mansouri, S. S., Karvelis, P., Georgoulas, G., & Nikolakopoulos, G. (2017). Remaining useful battery life prediction for UAVs based on machine learning. IFAC-PapersOnLine, 50(1), 4727-4732.

Meng, F., Zou, Q., Zhang, Z., Wang, B., Ma, H., Abdullah, H. M., ... & Mohamed, M. A. (2021). An intelligent hybrid wavelet-adversarial deep model for accurate prediction of solar power generation. Energy Reports, 7, 2155-2164.

Ng, M. F., Zhao, J., Yan, Q., Conduit, G. J., & Seh, Z. W. (2020). Predicting the state of charge and health of batteries using data-driven machine learning. Nature Machine Intelligence, 2(3), 161-170.

Oji, T., Zhou, Y., Ci, S., Kang, F., Chen, X., & Liu, X. (2021). Data-driven methods for battery soh estimation: Survey and a critical analysis. *Ieee Access*, *9*, 126903-126916.

Pan, D., Li, H., & Wang, S. (2022). Transfer learning-based hybrid remaining useful life prediction for lithium-ion batteries under different stresses. IEEE Transactions on Instrumentation and Measurement, 71, 1-10.

Pan, Y., Feng, X., Zhang, M., Han, X., Lu, L., & Ouyang, M. (2020). Internal short circuit detection for lithium-ion battery pack with parallel-series hybrid connections. Journal of cleaner production, 255, 120277.

Peng, Y., Hou, Y., Song, Y., Pang, J., & Liu, D. (2018). Lithium-ion battery prognostics with hybrid Gaussian process function regression. Energies, 11(6), 1420.

Pramanik, P. K. D., Pal, S., & Choudhury, P. (2019). Green and sustainable high-performance computing with smartphone crowd computing. Scalable Computing: Practice and Experience, 20(2), 259-284.

Qiu, X., Wu, W., & Wang, S. (2020). Remaining useful life prediction of lithium-ion battery based on improved cuckoo search particle filter and a novel state of charge estimation method. Journal of Power Sources, 450, 227700.

Rajendran, V., Tang, C., & van Paasschen, F. (2023, June). Analyzing synthetic datasets through the training and inference domain gap. In *Synthetic Data for Artificial Intelligence and Machine Learning: Tools, Techniques, and Applications* (Vol. 12529, pp. 203-207). SPIE.

Razavi-Far, R., Chakrabarti, S., Saif, M., & Zio, E. (2019). An integrated imputation-prediction scheme for prognostics of battery data with missing observations. Expert Systems with Applications, 115, 709-723.

Ren, L., Zhao, L., Hong, S., Zhao, S., Wang, H., & Zhang, L. (2018). Remaining useful life prediction for lithium-ion battery: A deep learning approach. Ieee Access, 6, 50587-50598.

Richardson, R. R., Osborne, M. A., & Howey, D. A. (2017). Gaussian process regression for forecasting battery state of health. Journal of Power Sources, 357, 209-219.

Rohr, S., Müller, S., Baumann, M., Kerler, M., Ebert, F., Kaden, D., & Lienkamp, M. (2017). Quantifying uncertainties in reusing lithium-ion batteries from electric vehicles. Procedia Manufacturing, 8, 603-610.

Rouhi Ardeshiri, R., & Ma, C. (2021). Multivariate gated recurrent unit for battery remaining useful life prediction: A deep learning approach. International Journal of Energy Research, 45(11), 16633-16648.

S.-J. Kwon, D. Han, J.H. Choi, J.-H. Lim, S.-E. Lee, J. Kim, J. Electroanal. Chem. 858 (2020).

Sanguesa, J. A., Torres-Sanz, V., Garrido, P., Martinez, F. J., & Marquez-Barja, J. M. (2021). A review on electric vehicles: Technologies and challenges. Smart Cities, 4(1), 372-404.

Sui, X., He, S., Vilsen, S. B., Meng, J., Teodorescu, R., & Stroe, D. I. (2021). A review of non-probabilistic machine learning-based state of health estimation techniques for Lithium-ion battery. *Applied Energy*, *300*, 117346.

Šeruga, D., Gosar, A., Sweeney, C. A., Jaguemont, J., Van Mierlo, J., & Nagode, M. (2021). Continuous modelling of cyclic ageing for lithium-ion batteries. Energy, 215, 119079.

Severson, K. A., Attia, P. M., Jin, N., Perkins, N., Jiang, B., Yang, Z., ... & Braatz, R. D. (2019). Data-driven prediction of battery cycle life before capacity degradation. Nature Energy, 4(5), 383-391.

Shen, D., Wu, L., Kang, G., Guan, Y., & Peng, Z. (2021). A novel online method for predicting the remaining useful life of lithium-ion batteries considering random variable discharge current. Energy, 218, 119490.

Shen, S., Sadoughi, M., Li, M., Wang, Z., & Hu, C. (2020). Deep convolutional neural networks with ensemble learning and transfer learning for capacity estimation of lithium-ion batteries. Applied Energy, 260, 114296.

Shi, D., Zhao, J., Eze, C., Wang, Z., Wang, J., Lian, Y., & Burke, A. F. (2023). Cloud-based artificial intelligence framework for battery management system. *Energies*, *16*(11), 4403.

Spell, G., Nadell, C., Bahhur, B., & Manser, K. (2023, June). Real-data performance evaluation of composite synthetic IR data. In *Synthetic Data for Artificial Intelligence and Machine Learning: Tools, Techniques, and Applications* (Vol. 12529, pp. 66-77). SPIE.

Sun, B., He, X., Zhang, W., Ruan, H., Su, X., & Jiang, J. (2020). Study of parameters identification method of li-ion battery model for EV power profile based on transient characteristics data. IEEE Transactions on Intelligent Transportation Systems, 22(1), 661-672.

Tagade, P., Hariharan, K. S., Ramachandran, S., Khandelwal, A., Naha, A., Kolake, S. M., & Han, S. H. (2020). Deep Gaussian process regression for lithium-ion battery health prognosis and degradation mode diagnosis. Journal of Power Sources, 445, 227281.

Tan, H., Zhang, H., Peng, J., Jiang, Z., & Wu, Y. (2019). Energy management of hybrid electric bus based on deep reinforcement learning in continuous state and action space. Energy Conversion and Management, 195, 548-560.

Tang, T., & Yuan, H. (2021). The capacity prediction of Li-ion batteries based on a new feature extraction technique and an improved extreme learning machine algorithm. Journal of Power Sources, 514, 230572.

Tang, X., Liu, K., Wang, X., Gao, F., Macro, J., & Widanage, W. D. (2020). Model migration neural network for predicting battery aging trajectories. IEEE Transactions on Transportation Electrification, 6(2), 363-374.

Tang, X., Wang, Y., Zou, C., Yao, K., Xia, Y., & Gao, F. (2019). A novel framework for Lithium-ion battery modeling considering uncertainties of temperature and aging. Energy conversion and management, 180, 162-170.

Taylor, L., & Nitschke, G. (2018, November). Improving deep learning with generic data augmentation. In 2018 IEEE symposium series on computational intelligence (SSCI) (pp. 1542-1547). IEEE.

Tian, H., Qin, P., Li, K., & Zhao, Z. (2020). A review of the state of health for lithium-ion batteries: Research status and suggestions. Journal of Cleaner Production, 261, 120813.

Vázquez-Canteli, J. R., & Nagy, Z. (2019). Reinforcement learning for demand response: A review of algorithms and modeling techniques. Applied energy, 235, 1072-1089.

Velho, R., Beirão, M., Calado, M. D. R., Pombo, J., Fermeiro, J., & Mariano, S. (2017). Management system for large li-ion battery packs with a new adaptive multistage charging method. Energies, 10(5), 605.

Vilsen, S. B., Sui, X., & Stroe, D. I. (2020, November). A Time-Varying Log-linear Model for Predicting the Resistance of Lithium-ion Batteries. In 2020 IEEE 9th International Power Electronics and Motion Control Conference (IPEMC2020-ECCE Asia) (pp. 1659-1666). IEEE.

Wang, C., Ning, Y., Li, X., & Li, H. (2020). A robust unscented kalman filter applied to ultra-wideband positioning. International Journal of Image and Data Fusion, 11(4), 308-330.

Wang, D., Yang, F., Tsui, K. L., Zhou, Q., & Bae, S. J. (2016). Remaining useful life prediction of lithium-ion batteries based on spherical cubature particle filter. *IEEE Transactions on Instrumentation and Measurement*, *65*(6), 1282-1291.

Wang, F. K., & Mamo, T. (2018). A hybrid model based on support vector regression and differential evolution for remaining useful lifetime prediction of lithium-ion batteries. Journal of Power Sources, 401, 49-54.

Wang, S. L., Tang, W., Fernandez, C., Yu, C. M., Zou, C. Y., & Zhang, X. Q. (2019). A novel endurance prediction method of series connected lithium-ion batteries based on the voltage change rate and iterative calculation. Journal of cleaner production, 210, 43-54.

Wang, S., Jin, S., Bai, D., Fan, Y., Shi, H., & Fernandez, C. (2021). A critical review of improved deep learning methods for the remaining useful life prediction of lithium-ion batteries. *Energy Reports*, *7*, 5562-5574.

Wang, Y., Pan, R., Yang, D., Tang, X., & Chen, Z. (2017). Remaining useful life prediction of lithium-ion battery based on discrete wavelet transform. Energy Procedia, 105, 2053-2058.

Wang, Z., Ma, Q., & Guo, Y. (2021, September). Remaining useful life prediction of lithium-ion batteries based on deep learning and soft sensing. In Actuators (Vol. 10, No. 9, p. 234). MDPI.

Wei, J., Dong, G., & Chen, Z. (2017). Remaining useful life prediction and state of health diagnosis for lithium-ion batteries using particle filter and support vector regression. IEEE Transactions on Industrial Electronics, 65(7), 5634-5643.

Wei, Z., He, H., Pou, J., Tsui, K. L., Quan, Z., & Li, Y. (2020). Signal-disturbance interfacing elimination for unbiased model parameter identification of lithium-ion battery. IEEE Transactions on Industrial Informatics, 17(9), 5887-5897.

Wu, J., Wei, Z., Liu, K., Quan, Z., & Li, Y. (2020). Battery-involved energy management for hybrid electric bus based on expert-assistance deep deterministic policy gradient algorithm. IEEE Transactions on Vehicular Technology, 69(11), 12786-12796.

Xiong, R., Li, L., Li, Z., Yu, Q., & Mu, H. (2018). An electrochemical model based degradation state identification method of Lithium-ion battery for all-climate electric vehicles application. Applied energy, 219, 264-275.

Xiong, R., Pan, Y., Shen, W., Li, H., & Sun, F. (2020). Lithium-ion battery aging mechanisms and diagnosis method for automotive applications: Recent advances and perspectives. Renewable and Sustainable Energy Reviews, 131, 110048.

Xiong, R., Tian, J., Shen, W., Lu, J., & Sun, F. (2023). Semi-supervised estimation of capacity degradation for lithium ion batteries with electrochemical impedance spectroscopy. Journal of Energy Chemistry, 76, 404-413.

Xu, B., Oudalov, A., Ulbig, A., Andersson, G., & Kirschen, D. S. (2016). Modeling of lithium-ion battery degradation for cell life assessment. IEEE Transactions on Smart Grid, 9(2), 1131-1140.

Xu, C., Zhang, F., Feng, X., Jiang, F., Ren, D., Lu, L., ... & Ouyang, M. (2021). Experimental study on thermal runaway propagation of lithium-ion battery modules with different parallel-series hybrid connections. Journal of Cleaner Production, 284, 124749.

Xue, Q., Shen, S., Li, G., Zhang, Y., Chen, Z., & Liu, Y. (2020). Remaining useful life prediction for lithium-ion batteries based on capacity estimation and box-cox transformation. IEEE Transactions on Vehicular Technology, 69(12), 14765-14779.

Y. Song, L. Li, Y. Peng, D. Liu, in: 2018 12th Int. Conf. Reliab. Maint. Saf., IEEE, 2018, pp. 317–322

Y. Zhang, R. Xiong, H. He, Z. Liu, in: 2017 Progn. Syst. Heal. Manag. Conf., IEEE, 2017, pp. 1–4

Yang, F., Wang, D., Xu, F., Huang, Z., & Tsui, K. L. (2020). Lifespan prediction of lithium-ion batteries based on various extracted features and gradient boosting regression tree model. Journal of Power Sources, 476, 228654.

Yang, H., Wang, P., An, Y., Shi, C., Sun, X., Wang, K., ... & Ma, Y. (2020). Remaining useful life prediction based on denoising technique and deep neural network for lithium-ion capacitors. ETransportation, 5, 100078.

Yang, Q., Li, J., Cao, W., Li, S., Lin, J., Huo, D., & He, H. (2020). An improved vehicle to the grid method with battery longevity management in a microgrid application. Energy, 198, 117374.

Yang, R., Xiong, R., Ma, S., & Lin, X. (2020). Characterization of external short circuit faults in electric vehicle Li-ion battery packs and prediction using artificial neural networks. Applied Energy, 260, 114253.

Yang, Z., Patil, D., & Fahimi, B. (2017). Online estimation of capacity fade and power fade of lithium-ion batteries based on input–output response technique. IEEE Transactions on Transportation Electrification, 4(1), 147-156.

Yang, Z., Patil, D., & Fahimi, B. (2018). Electrothermal modeling of lithium-ion batteries for electric vehicles. IEEE Transactions on Vehicular Technology, 68(1), 170-179.

Yin, L., Gao, Q., Zhao, L., & Wang, T. (2020). Expandable deep learning for real-time economic generation dispatch and control of three-state energies based future smart grids. Energy, 191, 116561.

Yin, Y., Hu, Y., Choe, S. Y., Cho, H., & Joe, W. T. (2019). New fast charging method of lithium-ion batteries based on a reduced order electrochemical model considering side reaction. Journal of Power Sources, 423, 367-379.

Yu, J., Yang, J., Wu, Y., Tang, D., & Dai, J. (2020). Online state-of-health prediction of lithium-ion batteries with limited labeled data. International Journal of Energy Research, 44(14), 11345-11360.

Yu, Y. (2020). *Social Media Usage in Crisis Communication: a Case Study of Samsung* (Master's thesis, Syracuse University).

Zhang, C., He, Y., & Yuan, L. (2019). Approach for Lithium-ion Battery RUL Prognostics Based on CPSO-RVM. *Journal of System Simulation*, *30*(5), 1935-1940.

Zhang, H., Mo, Z., Wang, J., & Miao, Q. (2019). Nonlinear-drifted fractional brownian motion with multiple hidden state variables for remaining useful life prediction of lithium-ion batteries. IEEE Transactions on Reliability, 69(2), 768-780.

Zhang, K., Patki, N., & Veeramachaneni, K. (2022). Sequential models in the synthetic data vault. arXiv preprint arXiv:2207.14406.

Zhang, M., Kang, G., Wu, L., & Guan, Y. (2022). A method for capacity prediction of lithium-ion batteries under small sample conditions. Energy, 238, 122094.

Zhang, S., Guo, X., & Zhang, X. (2020). Multi-objective decision analysis for data-driven based estimation of battery states: A case study of remaining useful life estimation. International Journal of Hydrogen Energy, 45(27), 14156-14173.

Zhang, S., Zhai, B., Guo, X., Wang, K., Peng, N., & Zhang, X. (2019). Synchronous estimation of state of health and remaining useful lifetime for lithium-ion battery using the incremental capacity and artificial neural networks. *Journal of Energy Storage*, *26*, 100951.

Zhang, Xiao-Qin, 2019. A novel endurance prediction method of series connected lithium-ion batteries based on the voltage change rate and iterative calculation. J. Cleaner Prod. 210, 43–54.

Zhang, Y., Tang, Q., Zhang, Y., Wang, J., Stimming, U., & Lee, A. A. (2020). Identifying degradation patterns of lithium ion batteries from impedance spectroscopy using machine learning. Nature communications, 11(1), 1706.

Zhang, Y., Xiong, R., He, H., & Pecht, M. (2019). Validation and verification of a hybrid method for remaining useful life prediction of lithium-ion batteries. Journal of Cleaner Production, 212, 240-249.

Zhang, Y., Xiong, R., He, H., & Pecht, M. G. (2018). Lithium-ion battery remaining useful life prediction with Box–Cox transformation and Monte Carlo simulation. IEEE Transactions on Industrial Electronics, 66(2), 1585-1597.

Zhang, Y., Xiong, R., He, H., & Pecht, M. G. (2018). Long short-term memory recurrent neural network for remaining useful life prediction of lithium-ion batteries. IEEE Transactions on Vehicular Technology, 67(7), 5695-5705.

Zhao, G., Zhang, G., Liu, Y., Zhang, B., & Hu, C. (2017, June). Lithium-ion battery remaining useful life prediction with deep belief network and relevance vector machine. In 2017 IEEE international conference on prognostics and health management (ICPHM) (pp. 7-13). IEEE.

Zhao, M., Zhong, S., Fu, X., Tang, B., Dong, S., & Pecht, M. (2020). Deep residual networks with adaptively parametric rectifier linear units for fault diagnosis. IEEE transactions on industrial electronics, 68(3), 2587-2597.

Zhao, Q., Qin, X., Zhao, H., & Feng, W. (2018). A novel prediction method based on the support vector regression for the remaining useful life of lithium-ion batteries. Microelectronics Reliability, 85, 99-108.

Zhao, S., Jiang, C., Zhang, Z., & Long, X. (2020). Robust remaining useful life estimation based on an improved unscented Kalman filtering method. Computer Modeling in Engineering & Sciences, 123(3), 1151-1173.

Zhao, Y., Stein, P., Bai, Y., Al-Siraj, M., Yang, Y., & Xu, B. X. (2019). A review on modeling of electro-chemo-mechanics in lithium-ion batteries. Journal of Power Sources, 413, 259-283.

Zheng, Y., Wu, H., Yi, W., Lai, X., Dai, H., Han, X., & Ouyang, M. (2020). A novel classification method of commercial lithium-ion battery cells based on fast and economic detection of self-discharge rate. Journal of power sources, 478, 229039.

Zhou, B., Cheng, C., Ma, G., & Zhang, Y. (2020, July). Remaining useful life prediction of lithium-ion battery based on attention mechanism with positional encoding. In IOP Conference Series: Materials Science and Engineering (Vol. 895, No. 1, p. 012006). IOP Publishing.

Zhou, Y., & Huang, M. (2016). Lithium-ion batteries remaining useful life prediction based on a mixture of empirical mode decomposition and ARIMA model. Microelectronics Reliability, 65, 265-273.

Zhou, Y., Huang, M., & Pecht, M. (2020). Remaining useful life estimation of lithium-ion cells based on k-nearest neighbor regression with differential evolution optimization. Journal of Cleaner Production, 249, 119409.

Zhou, Y., Huang, M., & Pecht, M. (2020). Remaining useful life estimation of lithium-ion cells based on k-nearest neighbor regression with differential evolution optimization. Journal of Cleaner Production, 249, 119409.

Zou, H., Tao, J., Elsayed, S. K., Elattar, E. E., Almalaq, A., & Mohamed, M. A. (2021). Stochastic multi-carrier energy management in the smart islands using reinforcement learning and unscented transform. International Journal of Electrical Power & Energy Systems, 130, 106988.

ANNEX A

Supervisor requirement evidence map

This annex makes the required assessment chain explicit and points to the original manuscript and implementation evidence. Page references are the printed page numbers in the original thesis, not the physical page number of this review edition.

Required element	What is demonstrated	Primary evidence
Problem solved	RUL prediction under ageing-dependent battery behaviour	Abstract; Introduction, pp. 3-4
Why relevant	Safety, availability, maintenance, and dependable power	Introduction, pp. 3-4
Suggested solution	Synthetic generation plus LSTM/XGBoost evaluation	Contribution, p. 6; Chapters 3 and 5
Why reasonable	Data-driven modelling, dependency-aware synthesis, sequential and tree baselines	Sections 1.2, 3.2-3.4, 5.2
Implementation	Device assignment, generation, cleaning, sequence preparation, training	Figures 9-14 and 25-29; notebook cells 4-28
Results	Correlation evidence, Frobenius norm, RMSE, MAE, R-squared	Tables 5-6; Figures 30-31; notebook outputs
Discussion	Fidelity strengths, observed differences, model comparison, limits	Section 3.4.4 and Annex D
Conclusion	LSTM is stronger in the recorded study; workflow supports RUL experimentation	Section 5.3, pp. 65-66
Annexes	Code listing, data sheet, result provenance, artifact manifest	Annexes B-E in this edition

ANNEX B

Dataset sheet and provenance

Repository snapshot

Artifact	Rows	Columns	SHA-256 (first 16)
real_battery_cycle_data.csv	15,064	11	36f4269f1e4b8191
processed_synthetic_battery_data.csv	3,093	11	343ec99cb9248b9f
RUL_Estimation_and_data_generation.ipynb	30 cells	-	84b7db40687b3c49
requirements.txt	10 packages	-	e6889d95f105c2ea

Dataset interpretation

- The real CSV snapshot contains 15,064 cycle-level rows and 10 named analytical variables plus an exported index column.
- The processed synthetic CSV snapshot contains 3,093 rows after filtering and reduction. The thesis reports 100,000 generated rows, while the supplied notebook cell is configured for 1,000,000. These are distinct from the retained output, but the raw-count mismatch must be corrected or explained.
- The notebook loads a file named Final Database.csv. The submission package should document that this corresponds to the supplied real_battery_cycle_data.csv snapshot or provide the original file under the notebook's expected name.
- The source data is described in the project materials as the public Kaggle Battery Remaining Useful Life (RUL) dataset. The final submission should retain a complete citation and access date.

Data fields

Field	Unit / role	Meaning in the workflow
Cycle_Index	cycle index	Progression of battery cycling
Discharge Time	seconds	Duration of discharge
Decrement 3.6-3.4V	seconds	Time for voltage to fall through the specified interval
Max. Voltage Dischar.	volts	Maximum discharge voltage
Min. Voltage Charg.	volts	Minimum charge voltage
Time at 4.15V	seconds	Duration at the specified charging voltage
Time constant current	seconds	Constant-current charging duration
Charging time	seconds	Total charging duration
Total time	seconds	Overall recorded cycle time
RUL	cycles / target	Remaining useful life prediction target

ANNEX C

Implementation code from the original notebook

The following listing reproduces selected original code cells without changing model logic. Long lines are visually wrapped for print. The complete executable source remains the supplied Jupyter notebook.

C.1 Device sequence assignment and synthesizer initialization

Original notebook cell 4

```
from sdv.single_table import GaussianCopulaSynthesizer
from sdv.metadata import SingleTableMetadata
# Initialize the 'DeviceID' column with zeros
data['DeviceID'] = 1

# Variable to hold the current device ID
current_device_id = 1

# Iterate through the rows to assign device IDs based on the logic described
for i in range(1, len(data)):
    # If the RUL at the current row is higher than the RUL at the previous row,
    # it indicates the start of a new device/cycle
    if data.iloc[i]['RUL'] > data.iloc[i - 1]['RUL']:
        current_device_id += 1
    data.at[data.index[i], 'DeviceID'] = current_device_id
data['DeviceID'][0]= 1
# Now, let's check the first few rows to verify the 'DeviceID' assignment
data.head()
```

C.2 Metadata, distribution choices, fitting, and raw sampling

Original notebook cell 5

```
from sdv.single_table import GaussianCopulaSynthesizer
from sdv.metadata import SingleTableMetadata

# Assuming 'data' is your DataFrame loaded from the CSV
metadata = SingleTableMetadata()
metadata.detect_from_dataframe(data)

# Instantiate the Gaussian Copula Synthesizer with your metadata
synthesizer = GaussianCopulaSynthesizer(
    metadata=metadata, # required
    enforce_min_max_values=True,
    numerical_distributions={
        'Discharge Time (s)': 'beta',
        'Total time (s)': 'uniform'
    },
    default_distribution='norm'
)


# Fit the synthesizer to your dataset
synthesizer.fit(data)

# Generate synthetic data, adjust num_rows as needed
synthetic_data = synthesizer.sample(num_rows=1000000)  # Generates 100 rows of synthetic data
sorted_synthetic_data = synthetic_data.sort_values(by=['DeviceID', 'RUL'], ascending=[True, False])
```

C.3 Synthetic-data filtering and monotonic RUL handling

Original notebook cell 6

```
# Filter rows where Decrement is positive
positive_decrement_df = sorted_synthetic_data[sorted_synthetic_data['Decrement 3.6-3.4V (s)'] > 0]

# Sort by DeviceID, Cycle_Index, and RUL in descending order
sorted_df = positive_decrement_df.sort_values(by=['DeviceID', 'Cycle_Index', 'RUL'], ascending=[True,
    True, False])

# Group by DeviceID and Cycle_Index and then randomly select one row per group
selected_df = sorted_df.groupby(['DeviceID', 'Cycle_Index']).apply(lambda x:
    x.sample(1)).reset_index(drop=True)
```

```
# Ensure that RUL values are decreasing for each DeviceID
def filter_decreasing_rul(group):
    group = group.sort_values(by='Cycle_Index')
    valid_rows = []
    last_rul = float('inf')
    for index, row in group.iterrows():
        if row['RUL'] < last_rul:
            valid_rows.append(row)
            last_rul = row['RUL']
    return pd.DataFrame(valid_rows)

final_df = selected_df.groupby('DeviceID').apply(filter_decreasing_rul).reset_index(drop=True)
```

C.4 Processed synthetic-data export

Original notebook cell 7

```
final_df.drop(['Unnamed: 0','DeviceID'],axis = 1).to_csv('synthetic_data.csv')
```

C.5 Correlation comparison and Frobenius norm

Original notebook cell 12

```
import numpy as np

# Calculate correlation matrices for both datasets
correlation_matrix_real = real_data.drop(['DeviceID'],axis = 1).iloc[:, 1:].corr()  # Exclude the
    Unnamed: 0 column
correlation_matrix_synthetic = synthetic_data.iloc[:, 1:].corr()

correlation_matrix_real_display = correlation_matrix_real
correlation_matrix_synthetic_display = correlation_matrix_synthetic

# Recalculate the Frobenius norm
frobenius_norm_recalculation = np.linalg.norm(correlation_matrix_real - correlation_matrix_synthetic)

(correlation_matrix_real_display, correlation_matrix_synthetic_display, frobenius_norm_recalculation)
```

C.6 Feature selection, scaling, sequence creation, and split

Original notebook cell 19

```
import pandas as pd
from sklearn.preprocessing import MinMaxScaler
from sklearn.model_selection import train_test_split
import numpy as np
# Compute the absolute value of correlation with the target
correlation_matrix = new_data.corr().abs()
correlation_with_target = correlation_matrix['RUL']

# Define a correlation threshold
threshold = 0.3  # You can adjust this value

# Find columns with correlation less than the threshold
columns_to_drop = correlation_with_target[correlation_with_target < threshold].index.tolist()

# Drop these columns from the features DataFrame
features = new_data.drop(columns=columns_to_drop + ['RUL'])
features_to_scale = new_data.columns.difference(['Cycle_Index', 'RUL'])
scaler = MinMaxScaler()
new_data[features_to_scale] = scaler.fit_transform(new_data[features_to_scale])

# Now proceed with the rest of your pipeline as before
look_back = 5
target = new_data['RUL']

def create_sequences(features, target, look_back=1):
    X, y = [], []
    for i in range(len(features) - look_back):
        x_sequence = features.iloc[i:(i + look_back)].values
        y_sequence = target.iloc[i + look_back]
        X.append(x_sequence)
        y.append(y_sequence)
    return np.array(X), np.array(y)

X, y = create_sequences(features, target, look_back)

# Split the data into training and testing sets
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

X_train.shape, X_test.shape
```

C.7 LSTM hyperparameter search

Original notebook cell 21

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense
from tensorflow.keras.optimizers import Adam
import numpy as np

# List of hyperparameters to try
lstm_units_options = [20, 50, 100]
optimizer_lr_options = [0.001, 0.01, 0.1]
batch_size_options = [16, 32, 64]
epochs_options = [5, 10, 30]

# Record the best score and corresponding hyperparameters
best_score = np.inf
best_params = {}

for lstm_units in lstm_units_options:
    for optimizer_lr in optimizer_lr_options:
        for batch_size in batch_size_options:
            for epochs in epochs_options:
                # Create the model with the current set of hyperparameters
                model = Sequential([
                    LSTM(lstm_units, input_shape=(X_train.shape[1], X_train.shape[2]),
                        return_sequences=True),
                    LSTM(lstm_units, return_sequences=False),
                    Dense(50, activation='relu'),
                    Dense(1)
                ])
                print(1)
                model.compile(loss='mean_squared_error', optimizer=Adam(learning_rate=optimizer_lr))

                # Fit the model
                history = model.fit(X_train, y_train, epochs=epochs, batch_size=batch_size,
                    validation_split=0.2, verbose=0)

                # Evaluate the model
                val_loss = np.min(history.history['val_loss'])  # Get the best validation loss achieved

                # Update the best parameters if the current model is better
                if val_loss < best_score:
                    best_score = val_loss
                    best_params = {
                        'lstm_units': lstm_units,
                        'optimizer_lr': optimizer_lr,
                        'batch_size': batch_size,
                        'epochs': epochs
                        }

# Print the best model's parameters and its score
print("Best Validation Loss: {:.4f}".format(best_score))
print("Best Parameters:")
print(best_params)
```

Recorded output

```
Best Validation Loss: 6689.5508
Best Parameters:
{'lstm_units': 50, 'optimizer_lr': 0.01, 'batch_size': 16, 'epochs': 30}
```

C.8 Final LSTM configuration and training

Original notebook cell 23

```
from keras.models import Sequential
from keras.layers import LSTM, Dense
from keras.optimizers import Adam

# Build the LSTM model
lstm_model = Sequential([
    LSTM(50, input_shape=(X_train.shape[1], X_train.shape[2]), return_sequences=True),
    LSTM(50, return_sequences=False),
    Dense(50, activation='relu'),
    Dense(1)
])
```

```
# Compile the model
lstm_model.compile(loss='mean_squared_error', optimizer=Adam(learning_rate=0.001))

# Train the model
lstm_history = lstm_model.fit(X_train, y_train, epochs=50, batch_size=32, validation_split=0.2,
    verbose=1)
```

C.9 LSTM evaluation metrics

Original notebook cell 24

```
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

# Make predictions
y_pred_lstm = lstm_model.predict(X_test)
n=len(X)
p=X.shape[1]
# Calculate metrics
rmse_lstm = np.sqrt(mean_squared_error(y_test, y_pred_lstm))
mae_lstm = mean_absolute_error(y_test, y_pred_lstm)
r2_lstm = r2_score(y_test, y_pred_lstm)
r2_adj_lstm = 1 - ((1-r2_lstm) * (n-1) / (n-p-1))  # Adjusted R² assuming p (number of features) is the
    number of inputs to the model

print(f"RMSE: {rmse_lstm}, MAE: {mae_lstm}, R²: {r2_lstm}, Adjusted R²: {r2_adj_lstm}")
```

Recorded output

```
RMSE: 85.40499393497447, MAE: 15.554181031141235, R²: 0.9308047894188476, Adjusted R²:
    0.9306952339872752
```

C.10 XGBoost grid search

Original notebook cell 25

```
from xgboost import XGBRegressor
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import make_scorer, mean_squared_error
import numpy as np

# Flatten the input data
X_train_flat = X_train.mean(axis=1)
X_test_flat = X_test.mean(axis=1)

# Define the model
xgb_model = XGBRegressor()

# Define the parameter grid
param_grid = {
    'n_estimators': [50, 100, 200],
    'learning_rate': [0.01, 0.05, 0.1],
    'max_depth': [3, 5, 7],
    'subsample': [0.8, 0.9, 1.0],
    'colsample_bytree': [0.8, 0.9, 1.0]
}

# Define a scorer, optional if you just want to use the default scorer
scorer = make_scorer(mean_squared_error, greater_is_better=False)

# Setup the GridSearchCV
grid_search = GridSearchCV(estimator=xgb_model, param_grid=param_grid, scoring=scorer, cv=3, verbose=2,
    n_jobs=-1)

# Perform the grid search
grid_result = grid_search.fit(X_train_flat, y_train)

# Best parameters and best score
print("Best: %f using %s" % (grid_result.best_score_, grid_result.best_params_))

# Optional: Extract results for further analysis
means = grid_result.cv_results_['mean_test_score']
stds = grid_result.cv_results_['std_test_score']
params = grid_result.cv_results_['params']
for mean, stdev, param in zip(means, stds, params):
    print("%f (%f) with: %r" % (mean, stdev, param))
```

Recorded output

```
Best: -9708.894549 using {'colsample_bytree': 1.0, 'learning_rate': 0.05, 'max_depth': 3,
    'n_estimators': 100, 'subsample': 0.9}
```

C.11 Final XGBoost configuration and evaluation

Original notebook cell 26

```
from xgboost import XGBRegressor

# Calculate the average of features across the time steps for each sequence
X_train_flat = X_train.mean(axis=1)
X_test_flat = X_test.mean(axis=1)

# Initialize and train the XGBoost model
xgb_model = XGBRegressor(n_estimators=100, learning_rate=0.05, max_depth=5, random_state=42)
xgb_model.fit(X_train_flat, y_train)

# Predict on test data
y_pred_xgb = xgb_model.predict(X_test_flat)

# Calculate RMSE, MAE, R², and adjusted R² for XGBoost model
rmse_xgb = np.sqrt(mean_squared_error(y_test, y_pred_xgb))
mae_xgb = mean_absolute_error(y_test, y_pred_xgb)
r2_xgb = r2_score(y_test, y_pred_xgb)
n, p = X_test_flat.shape[0], X_test_flat.shape[1]  # n is number of samples, p is number of features
r2_adj_xgb = 1 - ((1-r2_xgb) * (n-1) / (n-p-1))

print(f"RMSE: {rmse_xgb}, MAE: {mae_xgb}, R²: {r2_xgb}, Adjusted R²: {r2_adj_xgb}")
```

Recorded output

```
RMSE: 87.13293028357333, MAE: 24.759920030596884, R²: 0.9279765132623617, Adjusted R²:
    0.9276329990171902
```

C.12 Actual-versus-predicted plot

Original notebook cell 27

```
import matplotlib.pyplot as plt

plt.figure(figsize=(12, 6))
plt.scatter(y_test, y_pred_xgb, alpha=0.3, label='XGBoost Predictions')
plt.scatter(y_test, y_pred_lstm, alpha=0.3, label='LSTM Predictions')
plt.title('Actual vs. Predicted RUL')
plt.xlabel('Actual RUL')
plt.ylabel('Predicted RUL')
plt.legend()
plt.grid(True)
plt.show()
```

C.13 Performance comparison plot

Original notebook cell 28

```
import numpy as np

# Assuming you have calculated these metrics already
metrics = {
    'RMSE': [rmse_xgb, rmse_lstm],
    'MAE': [mae_xgb, mae_lstm],
    'R²': [r2_xgb*100, r2_lstm*100],
    'Adjusted R²': [r2_adj_xgb*100, r2_adj_lstm*100]
}
metrics_df = pd.DataFrame(metrics, index=['XGBoost', 'LSTM'])

metrics_df.plot(kind='bar', figsize=(12, 6))
plt.title('Comparison of Performance Metrics')
plt.ylabel('Metric Value')
plt.xticks(rotation=0)
plt.grid(True, linestyle='--')
plt.show()
```

ANNEX D

Results provenance and reproducibility audit

Recorded model outputs

Evidence	XGBoost	LSTM	Provenance
RMSE	87.132930	85.404994	Notebook cells 24 and 26
MAE	24.759920	15.554181	Notebook cells 24 and 26
R-squared	0.9279765	0.9308048	Notebook cells 24 and 26
Adjusted R-squared	0.9276330	0.9306952	Stored notebook outputs

Configuration trace

Stage	Search result stored in notebook	Configuration used for recorded final model
LSTM	50 units; learning rate 0.01; batch 16; 30 epochs	50 units; learning rate 0.001; batch 32; 50 epochs
XGBoost	depth 3; 100 estimators; learning rate 0.05; subsample 0.9; colsample 1.0	depth 5; 100 estimators; learning rate 0.05; random_state 42

The final configurations are not identical to the best stored search configurations. The thesis should therefore describe the metrics as results of the final recorded models, not as metrics of the stated best search candidates, unless the author reruns and documents the selected settings.

Dataset snapshot comparison

Measure	Published thesis value	Current CSV snapshot audit
Raw synthetic sample count	100,000	Notebook cell 5 requests 1,000,000; processed CSV retains 3,093
Correlation-matrix Frobenius norm	1.321	1.242
Cycle_Index vs RUL (real)	-0.999	-1.000
Cycle_Index vs RUL (synthetic)	-0.999	-1.000
Discharge Time vs RUL (synthetic)	0.236	0.209

The supplied processed CSV snapshot does not reproduce every published correlation value exactly. This is evidence that the snapshot may differ from the exact dataset state used for the thesis tables. The final archive should preserve the exact input and processed output used to generate the published results.

ANNEX D

Method limitations requiring explicit discussion

These points are visible in the supplied notebook and should be described as limitations or corrected through a documented rerun. No replacement metric is introduced here.

Observed implementation	Why it matters	Defensible treatment
MinMaxScaler is fitted before the train-test split	Information from held-out rows can influence scaling	Rerun with scaler fitted only on training data, or state this as a limitation
train_test_split is random across generated sequences	Adjacent cycles or the same device may appear in both partitions	Prefer device-level or chronological splitting for a stronger generalisation claim
The thesis reports 100,000 generated rows, notebook cell 5 requests 1,000,000, and the processed snapshot retains 3,093	The generation setting and final retained count are otherwise ambiguous	Reconcile the notebook setting, then state requested, generated, and retained counts separately
Search optima and final model settings differ	The tuning claim and reported metrics may refer to different configurations	Document the selection rationale or rerun using the selected optimum
One dataset is used	External validity is limited	Limit conclusions to the studied dataset and propose external validation

Recommended discussion wording boundary

A defensible conclusion is that the recorded LSTM implementation performed slightly better than the recorded XGBoost implementation on the supplied split and metrics. It is not yet defensible to claim universal superiority, production readiness, or cross-dataset generalisation.

No fabricated correction

Do not replace any value in the thesis merely to match this audit. A numerical correction requires a reproducible rerun from the exact input dataset, saved random seeds, preprocessing state, model configuration, and evaluation partition.

ANNEX E

Submission manifest and running instructions

Minimum electronic submission package

Item	Required file	Purpose
Thesis	Nayeem-Faisal-Thesis-Supervisor-Review-Edition.pdf	Readable report with mapped requirements and annexes
Notebook	notebooks/RUL_Estimation_and_data_generation.ipynb	Original implementation and stored outputs
Real data	data/real_battery_cycle_data.csv	Input cycle-level battery data snapshot
Synthetic data	data/processed_synthetic_battery_data.csv	Retained post-processing output
Dependencies	requirements.txt	Python package list
Readme	A corrected Bachelor-thesis README	Run order, data provenance, limitations, and artifact hashes

Recorded execution sequence

- Create a clean Python environment.
- Install the packages listed in requirements.txt.
- Place the exact real dataset at the path expected by the notebook, or update the first loading cell and document the change.
- Run the notebook from the first cell to the last cell and save all outputs.
- Archive the executed notebook, exact input data, processed output, package versions, random seeds, and generated figures together.

Dependency list

```
pandas
numpy
matplotlib
seaborn
scikit-learn
tensorflow
keras
xgboost
sdv
jupyter
```

Final integrity check

Before submission, Nayeem Faisal should confirm that every appendix file belongs to his original project, that the exact dataset used for the published results is included, and that the supervisor approves the format and disclosure of this editorial supplement.

TECHNISCHE HOCHSCHULE AUGSBURG

Development and Evaluation of Machine Learning Models for Lithium-Ion Battery Remaining Useful Life Prediction

AUTHOR'S DECLARATION

Declaration concerning the final project

I herewith affirm that I have drafted the submitted dissertation independently and have not used any other sources or resources than those declared therein. Sources quoted literally or substantively have been cited according to the acknowledged rules of scientific publication. Furthermore, I declare that the dissertation has not been submitted elsewhere. I have read and understood the explanatory note on fraud in examination procedures at the University of Applied Sciences Augsburg. I affirm that the submitted dissertation does not include any plagiarism or any texts or figures created by other persons acting on my instructions.

INDEPENDENT AUTHOR

NAYEEM FAISAL

Bachelor of Science - International Information Systems

Augsburg, 27 October 2026

Signature - Nayeem Faisal